



CPE 232 Data Models
Predicting Second-Hand Car Prices

จัดทำโดย

นายกรวิชญ์	ฮิคารุ	คุรากาเกะ	66070501001
นายพงศ์ภรณ์		อินทร์พันธุ์	66070501035
นายพันธุ์ธัช		เจียมจรัสโชติ	66070501038
นายภูบดินทร์		รามศิริ	66070501041
นายภูรินทร์		จันทพุด	66070501043
นายพงศ์วี		ศรีเพชรพงศ์	65070507218
นายสาวธวัชรรัตน์		โรจน์อมรรัตน์	65070507234

เสนอ

ด.ร. สัณณูสิริ ธารประดับ

สารบัญ

บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของโครงการ	2
1.4 เครื่องมือที่ใช้ในการพัฒนา	3
บทที่ 2 การสำรวจข้อมูล	4
2.1 คำอธิบายข้อมูล	4
2.2 จำนวนรายการข้อมูล	4
2.3 หมวดหมู่ของข้อมูล	4
2.4 คุณภาพของข้อมูล	5
บทที่ 3 การเตรียมและการจัดการข้อมูล	6
3.1 การโหลด libraries และ Dataset	6
3.2 การจัดการข้อมูลก่อนนำไปใช้	6
บทที่ 4 วิธีการสร้างแบบจำลองและผลลัพธ์	35
4.1 การนำเข้าไลบรารี	35
4.2 การฝึกโมเดล	35
บทที่ 5 วิเคราะห์ผล	44
5.1 การเปรียบเทียบผลลัพธ์ระหว่างแบบจำลองที่ปรับและไม่ปรับค่าพารามิเตอร์	44
5.2 การเปรียบเทียบประสิทธิภาพของ model แต่ละประเภท	45
บทที่ 6 สรุปผล	46
6.1 การเตรียมและการจัดการข้อมูล	46
6.2 การสร้างแบบจำลอง	46

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

รถยนต์เป็นหนึ่งในรูปแบบการเดินทางที่สำคัญทั่วโลกมาเป็นเวลาหลายทศวรรษ โดยตลาดรถยนต์มือสองเป็นหนึ่งในตลาดที่สำคัญของอุตสาหกรรมยานยนต์ อย่างไรก็ตาม ตลาดรถยนต์มือสองยังคงประสบปัญหาการตั้งราคาขายที่สูงเกินจริง โดยเฉพาะในกรณีที่ผู้ขายบางรายฉวยโอกาสจากความรู้ที่จำกัดของผู้ซื้อเกี่ยวกับราคาซื้อขายของรถยนต์มือสองมาเป็นช่องโหว่ในการหากำไร ซึ่งการตั้งราคาที่สูงเกินจริงของผู้ขายอาจเกิดจากความตั้งใจหรือการขาดความโปร่งใส ซึ่งอาจทำให้ผู้ซื้อสูญเสียเงินมากกว่าที่ควรจะเป็น และลดความเชื่อมั่นของตลาดรถยนต์มือสองลงอย่างมาก

เราจึงได้เลือกสร้างแบบจำลองสำหรับการคาดการณ์ราคารถยนต์มือสองเพื่อช่วยสร้างความตระหนักรู้ให้กับลูกค้าและป้องกันไม่ให้เกิดการตั้งราคาขายที่สูงเกินความเป็นจริงจากเต็นท์รถหรือผู้ขายรายบุคคล ด้วยการวิเคราะห์จากปัจจัยต่าง ๆ เช่น ราคา ระยะทางการใช้งาน (odometer) สภาพรถ ยี่ห้อ รุ่น ปีที่ผลิต และประเภทเชื้อเพลิง เป็นต้น โดยผู้จัดทำตั้งเป้าที่จะให้ผลการประเมินราคาที่แม่นยำจากข้อมูลที่เป็นกลาง เพื่อส่งเสริมให้ผู้บริโภคสามารถตัดสินใจได้โดยมีข้อมูลประกอบ และช่วยส่งเสริมให้เกิดการซื้อขายที่เป็นธรรมและโปร่งใสในตลาดรถยนต์มือสอง

1.2 วัตถุประสงค์

1. เพื่อพัฒนาแบบจำลองสำหรับการคาดการณ์ราคารถยนต์มือสอง โดยใช้เทคนิคการเรียนรู้ของเครื่อง (Machine Learning)
2. เพื่อให้ผู้ใช้งานสามารถเข้าถึงราคาประเมินรถยนต์มือสองที่ตนสนใจ เพื่อใช้เป็นข้อมูลประกอบการตัดสินใจ และลดโอกาสในการถูกตั้งราคาขายที่เกินจริง
3. เพื่อทดลองและเปรียบเทียบประสิทธิภาพของโมเดลการเรียนรู้ โดยประเมินจากตัวชี้วัดทางสถิติ เช่น ค่า MAE, RMSE และ R^2 เพื่อหาโมเดลที่เหมาะสมที่สุดสำหรับการคาดการณ์ราคารถยนต์มือสองในบริบทของข้อมูลจริง

1.3 ขอบเขตของโครงการ

โครงการนี้มีขอบเขตในการพัฒนาระบบคาดการณ์ราคารถยนต์มือสอง โดยใช้เทคนิคการเรียนรู้ของเครื่อง (Machine Learning) โดยเน้นการทดลองเพื่อเปรียบเทียบประสิทธิภาพของโมเดล 4 ประเภท ได้แก่ Linear Regression, Ridge Regression, Random Forest และ XGBoost โดยเปรียบเทียบระหว่างชุดที่ไม่มีการ tuning และมีการ tuning เพื่อหาระบบที่มีประสิทธิภาพ

ในการทดลองได้ใช้ข้อมูลจาก Used Cars Dataset ซึ่งประกอบด้วยข้อมูลจำนวนทั้งสิ้น 426,880 รายการ ซึ่งแบ่งออกเป็น 26 หมวดหมู่ โดยการเปรียบเทียบประสิทธิภาพของโมเดล เปรียบเทียบจากการคำนวณค่าชี้วัดทั้ง 4 แบบ ดังต่อไปนี้

1. R^2 (R-squared หรือ Coefficient of Determination)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

2. RMSE (Root Mean Squared Error)

$$RSME = \frac{\sqrt{\sum_{i=1}^N (y_i - \hat{y}_i)^2}}{N}$$

3. MAE (Mean Absolute Error)

$$MAE = \frac{1}{N} \sum_i^N (|y_i - \hat{y}_i|)$$

4. PMAE (%) (Percentage MAE หรือ MAE แบบเปอร์เซ็นต์)

$$PMAE = \frac{100\%}{N} \sum_i^N \frac{y_i - \hat{y}_i}{y_i}$$

1.4 เครื่องมือที่ใช้ในการพัฒนา

โครงการนี้ได้ใช้เครื่องมือและไลบรารีต่าง ๆ ดังนี้:

1. Pandas: ไลบรารีสำหรับการจัดการและวิเคราะห์ข้อมูลในรูปแบบตาราง (DataFrame) ใช้งานง่าย และทรงพลังในการจัดการข้อมูลที่มีโครงสร้าง เช่น ไฟล์ CSV หรือ SQL table
2. Numpy: ไลบรารีสำหรับการคำนวณเชิงตัวเลขที่มีประสิทธิภาพสูง โดยเฉพาะกับอาร์เรย์หลายมิติ (ndarray) และฟังก์ชันคณิตศาสตร์ต่างๆ
3. Matplotlib: ไลบรารีสำหรับสร้างกราฟและแผนภูมิทางสถิติ
4. Scikit-learn: ไลบรารี Machine Learning ที่ครอบคลุมการเรียนรู้ของเครื่องทั้ง Supervised และ Unsupervised เช่น การจัดกลุ่ม การถดถอย การจำแนก
5. Warnings: ไลบรารีมาตรฐานของ Python สำหรับแสดงคำเตือน (warning) แทนการหยุดโปรแกรมทันที ช่วยให้นักพัฒนาทราบปัญหาที่อาจเกิดขึ้น
6. fuzzywuzzy: ไลบรารีสำหรับการจับคู่อักขรแบบไม่เป๊ะ (fuzzy matching) เช่น การเปรียบเทียบคำที่พิมพ์ผิดหรือไม่ตรงกัน 100% โดยใช้ Levenshtein Distance
7. tqdm: ไลบรารีสำหรับแสดงแถบ Progress bar ใช้งานง่ายและมีประโยชน์มากในการติดตามความเร็วการประมวลผล
8. seaborn: ไลบรารีย่อยของ Matplotlib ที่ออกแบบมาเพื่อการสร้างกราฟสถิติที่ดีและใช้งานง่าย เช่น heatmap, distribution plot
9. joblib: ไลบรารีสำหรับการจัดเก็บและโหลดอ็อบเจกต์ Python เช่น โมเดล Machine Learning ที่ฝึกไว้แล้ว
10. Kneed: ไลบรารีสำหรับทำ Elbow Method หาจำนวนคลัสเตอร์ที่เหมาะสมใน K-Means
11. scipy.stats: โมดูลย่อยของ SciPy สำหรับการคำนวณทางสถิติ เช่น ค่าสหสัมพันธ์ การแจกแจงความน่าจะเป็น และการทดสอบสมมุติฐาน
12. mpl_toolkits: การแสดงผล 3 มิติ
13. xgboost: ไลบรารีสำหรับการเรียนรู้ของเครื่องแบบ Gradient Boosting ที่มีประสิทธิภาพสูง
14. optuna: ไลบรารีสำหรับการทำ hyperparameter tuning อย่างมีประสิทธิภาพ โดยใช้แนวคิดของ optimization เพื่อค้นหาค่าที่ดีที่สุดของโมเดล

บทที่ 2

การสำรวจข้อมูล

ผู้จัดทำได้ทำการสำรวจและวิเคราะห์ข้อมูลจากชุดข้อมูลชื่อ Used Cars Dataset ซึ่งเป็นชุดข้อมูลจาก Craigslist ซึ่งเป็นแหล่งรวมประกาศขายรถยนต์มือสองที่ใหญ่ที่สุดในโลก โดยมีจุดประสงค์เพื่อทำความเข้าใจลักษณะของข้อมูลก่อนการนำไปประมวลผลและฝึกโมเดล โดยมีรายละเอียดดังนี้

2.1 คำอธิบายข้อมูล

Craigslist เป็นแหล่งรวมประกาศขายรถยนต์มือสองที่ใหญ่ที่สุดในโลก อย่างไรก็ตาม การรวบรวมข้อมูลรถยนต์ทั้งหมดให้อยู่ในที่เดียวกันนั้นเป็นเรื่องที่ยาก ผู้จัดทำข้อมูลนี้ได้สร้างตัวดึงข้อมูล (scraper) สำหรับโครงการในโรงเรียน และได้ขยายผลงานนี้ต่อมาเพื่อสร้างชุดข้อมูลนี้ ซึ่งประกอบด้วยรายการรถยนต์มือสองทั้งหมดในสหรัฐอเมริกาที่อยู่บน Craigslist ทำให้ข้อมูลชุดนี้ถูกดึงใหม่ในทุก ๆ ไม่กี่เดือน โดยประกอบด้วยข้อมูลสำคัญที่ Craigslist ให้ไว้เกี่ยวกับการขายรถยนต์ เช่น ราคา สภาพรถ ผู้ผลิต พิกัดละติจูด/ลองจิจูด และหมวดหมู่อื่น ๆ อีก 18 หมวดหมู่ โดยสำหรับโปรเจกต์ด้านการเรียนรู้ของเครื่อง (ML) แนะนำให้ทำการแปลงฟีเจอร์จากข้อมูลตำแหน่ง เช่น ค่าละติจูด/ลองจิจูด และหากต้องการข้อมูลจากรายการเก่า สามารถตรวจสอบจากเวอร์ชันก่อนหน้าของชุดข้อมูลได้

2.2 จำนวนรายการข้อมูล

ข้อมูลจากชุดข้อมูล Used Cars Dataset ประกอบด้วยข้อมูลจำนวนทั้งสิ้นประมาณ 426,880 รายการ

2.3 หมวดหมู่ของข้อมูล

ข้อมูลจากชุดข้อมูล Used Cars Dataset ประกอบด้วย 26 หมวดหมู่ ได้แก่

- | | | | |
|----------------|----------------|---------------|----------------|
| ● id | ● model | ● VIN | ● county |
| ● url | ● condition | ● drive | ● state |
| ● region | ● cylinders | ● size | ● lat |
| ● region_url | ● fuel | ● type | ● long |
| ● price | ● odometer | ● paint_color | ● posting_date |
| ● year | ● title_status | ● image_url | |
| ● manufacturer | ● transmission | ● description | |

2.4 คุณภาพของข้อมูล

จากข้อมูลพบว่ามีปัญหาที่ต้องได้รับการแก้ไขก่อนนำไปใช้ ดังนี้

- Missing value: มีข้อมูลข้อมูลขาดหาย
- Out of date data (ex. url): มีข้อมูลที่ล้าสมัย เช่น url
- Outlier (ex. price = 0): มีค่าผิดปกติหรือค่านอกเกณฑ์ เช่น ราคา = 0
- 'other' values: มีข้อมูลประเภทอื่น ๆ ที่ไม่ได้ระบุประเภท เช่น ค่า 'other'
- Irregular model, manufacturer names: ชื่อรุ่นหรือผู้ผลิตที่ไม่อยู่ในรูปแบบเดียวกัน เช่น มีการเขียนชื่อเป็นตัวย่อ

บทที่ 3

การเตรียมและการจัดการข้อมูล

3.1 การโหลด libraries และ Dataset

```
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

df_car_sales = pd.read_csv('../dataset/vehicles.csv')
df_car_details = pd.read_json('../dataset/vehicle_details.json')
```

3.2 การจัดการข้อมูลก่อนนำไปใช้

3.2.1 การเตรียมข้อมูล (Preparation)

- ลบ column ที่ไม่เกี่ยวข้อง

```
drop_columns = ['id', 'url', 'region_url', 'VIN', 'image_url', 'description',
                'county', 'lat', 'long']

df.drop(columns = drop_columns, inplace = True)

df.info()
```

ผลลัพธ์หลังลบ

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 426880 entries, 0 to 426879
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   region                426880 non-null  object
1   price                 426880 non-null  int64
2   year                  425675 non-null  float64
3   manufacturer          409234 non-null  object
4   model                 421603 non-null  object
5   condition             252776 non-null  object
6   cylinders             249202 non-null  object
7   fuel                  423867 non-null  object
8   odometer              422480 non-null  float64
9   title_status          418638 non-null  object
10  transmission          424324 non-null  object
11  drive                 296313 non-null  object
12  size                  120519 non-null  object
13  type                  334022 non-null  object
14  paint_color           296677 non-null  object
15  state                 426880 non-null  object
16  posting_date          426812 non-null  object
dtypes: float64(2), int64(1), object(14)
memory usage: 55.4+ MB
```

- แปลงรูปแบบวันที่และเวลา

```
df["posting_date"] = pd.to_datetime(df["posting_date"], errors="coerce", utc=True)
```


3.2.2 การทำความสะอาดข้อมูล (cleaning)

- ลบ row ที่มีค่าที่หายไป (missing values) มากกว่า 1 ค่า

```

# Drop rows that contain at least one NaN value
print(f"Remaining records before dropping rows with any NaN >> {df.shape[0]}")
df_cleanwdrop = df[df.notna().all(axis=1)]

# Show result
print(f"Remaining records after dropping rows with any NaN >>
{df_cleanwdrop.shape[0]}")

```

- เก็บค่าไว้ในตัวแปร df_cleanwdrop โดยเก็บเฉพาะแถวที่ไม่ซ้ำและแสดงผลลัพธ์

```

df_cleanwdrop = df_cleanwdrop[~(df_cleanwdrop.duplicated())].copy()
df_cleanwdrop.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 79193 entries, 31 to 426836
Data columns (total 17 columns):
#   Column          Non-Null Count  Dtype
---  -
0   region          79193 non-null  object
1   price           79193 non-null  int64
2   year            79193 non-null  float64
3   manufacturer    79193 non-null  object
4   model           79193 non-null  object
5   condition       79193 non-null  object
6   cylinders       79193 non-null  object
7   fuel            79193 non-null  object
8   odometer        79193 non-null  float64
9   title_status    79193 non-null  object
10  transmission    79193 non-null  object
11  drive           79193 non-null  object
12  size            79193 non-null  object
13  type            79193 non-null  object
14  paint_color     79193 non-null  object
15  state           79193 non-null  object
16  posting_date    79193 non-null  datetime64[ns, UTC]
dtypes: datetime64[ns, UTC](1), float64(2), int64(1), object(13)
memory usage: 10.9+ MB

```

- สร้าง function ต่าง ๆ

- Function สำหรับจับคู่ข้อมูลยี่ห้อรถ (manufacturer) และรุ่นรถ (model) จาก df_data กับอีก df_details โดยใช้ fuzzy matching

```
from fuzzywuzzy import process
from tqdm.auto import tqdm
import re
fuzzy_cache = {}

# Use a dictionary to store previously seen matches

def get_matched_models(df_data, df_details):
    """
    Perform fuzzy matching to match makes and models between sample_df_sales and
    df_details.

    Parameters:
        sample_df_sales (pd.DataFrame): The sales dataframe containing manufacturer and
        model information.
        df_details (pd.DataFrame): The details dataframe containing make and model
        information.

    Returns:
        pd.DataFrame: The updated sample_df_sales dataframe with matched_make and
        matched_model columns.
    """

    def get_closest_match(row, column, choices):
        if pd.isnull(row[column]):
            return None

        value = str(row[column]).strip().lower()

        # Remove all non-alphanumeric characters for processing
        processed_value = re.sub(r'^a-z0-9', '', value)

        # If the processed value is empty, skip matching
        if not processed_value:
            return None

        if value in fuzzy_cache:
            return fuzzy_cache[value]

        match, score = process.extractOne(value, choices)
        result = match if score > 80 else None
        fuzzy_cache[value] = result
        return result

    # Apply fuzzy matching for manufacturer first
    df_details['make'] = df_details['make'].str.lower()

    # Add a progress bar to the process
    tqdm.pandas(desc="Matching manufacturers")
    df_data['matched_make'] = df_data.progress_apply(
        lambda row: get_closest_match(row, 'manufacturer',
        df_details['make'].unique()), axis=1
    )

    # Filter df_details to only include rows with the matched manufacturer
    def filter_models(row):
        if pd.isnull(row['matched_make']):
            return np.array([]) # Return an empty NumPy array
        return df_details[df_details['make'] == row['matched_make']]
    ['model'].unique()

    # Apply fuzzy matching for model based on the filtered models
    def get_closest_model(row):
        models = filter_models(row)
        if models.size == 0: # Explicitly check if the array is empty
            return None
        return get_closest_match(row, 'model', models)

    # Add a progress bar to the process
    tqdm.pandas(desc="Matching models")
    df_data['matched_model'] = df_data.progress_apply(get_closest_model, axis=1)

    # Calculate the number of matched and unmatched rows
    matched_count = df_data['matched_model'].notnull().sum()
    print(f"Matched: {matched_count} out of {len(df_data)} rows")

    return df_data
```

- Function สำหรับเติมค่าที่หายไปด้วย mean และ mode โดยใช้ข้อมูลที่ขึ้นกับ
ยี่ห้อ รุ่น และปีของรถ กับข้อมูลที่ไม่ขึ้นกับยี่ห้อ รุ่น และปีของรถ

```
from sklearn.cluster import KMeans
from sklearn.impute import SimpleImputer

def fill_model_deps(data_df, deps):
    """
    Fill missing values in model-dependent columns using the mode of data
    grouped by various keys.

    Parameters:
    df (pd.DataFrame): The dataframe to process.
    columns (list): List of model-dependent columns to fill.

    Returns:
    pd.DataFrame: The dataframe with missing values filled.
    """
    # Fill missing values grouped by manufacturer, model, and year
    for col in deps:
        data_df[col] = data_df.groupby(['manufacturer', 'model', 'year'])
        [col].transform(
            lambda x: x.fillna(x.mode().iloc[0]) if not x.mode().empty else x
        )

    # Fill missing values grouped by manufacturer and model
    for col in deps:
        data_df[col] = data_df.groupby(['manufacturer', 'model'])
        [col].transform(
            lambda x: x.fillna(x.mode().iloc[0]) if not x.mode().empty else x
        )

    # Fill missing values grouped by matched_make, matched_model, and year
    for col in deps:
        data_df[col] = data_df.groupby(['matched_make', 'matched_model',
        'year'])
        [col].transform(
            lambda x: x.fillna(x.mode().iloc[0]) if not x.mode().empty else x
        )

    # Fill missing values grouped by matched_make and matched_model
    for col in deps:
        data_df[col] = data_df.groupby(['matched_make', 'matched_model'])
        [col].transform(
            lambda x: x.fillna(x.mode().iloc[0]) if not x.mode().empty else x
        )

    return data_df

def fill_non_deps(data_df, deps):
    # Define model-dependent columns

    # Select columns to impute (excluding model-dependent columns)
    columns_to_impute = [col for col in data_df.columns if col not in deps]

    # Impute missing values using K-Means clustering

    # Select numeric columns for clustering
    numeric_cols = data_df.select_dtypes(include=[np.number]).columns
    data_for_clustering = data_df[numeric_cols]

    # Impute missing values with the mean before clustering
    imputer = SimpleImputer(strategy='mean')
    data_imputed = imputer.fit_transform(data_for_clustering)

    # Apply K-Means clustering
    kmeans = KMeans(n_clusters=data_df['type'].nunique(), random_state=42)
    clusters = kmeans.fit_predict(data_imputed)

    # Add cluster labels to the dataframe
    data_df['cluster'] = clusters

    # Impute missing values in the original dataframe using cluster means
    for col in columns_to_impute:
        if data_df[col].dtype in [np.float64, np.int64]: # Numeric columns
            data_df[col] = data_df.groupby('cluster')[col].transform(
                lambda x: x.fillna(x.mean())
            )
        elif data_df[col].dtype == object: # Categorical columns
            data_df[col] = data_df.groupby('cluster')[col].transform(
                lambda x: x.fillna(x.mode().iloc[0]) if not x.mode().empty else x
            )

    # Drop the cluster column
    data_df.drop(columns=['cluster'], inplace=True)

    return data_df
```

- Function สำหรับเติมค่าที่หายไป โดยใช้ข้อมูลจาก df_car_details

```
def fill_details(df_data, df_details):
    """
    Fill missing values in target columns of df_data using corresponding source
    columns from df_details,
    and analyze categorical columns in the updated dataframe.

    Parameters:
    df_data (pd.DataFrame): The sales dataframe.
    df_details (pd.DataFrame): The details dataframe.
    columns_to_fill (dict): A dictionary where keys are target columns in df_data
    and values are source columns in df_details.

    Returns:
    pd.DataFrame: The updated dataframe with filled values.
    """
    columns_to_fill = {
        'cylinders': 'cylinders_y',
        'fuel': 'fueltype1',
        'transmission': 'transy',
        'type': 'vclass'
    }

    # Convert matching columns to lowercase for consistent comparison
    df_data['matched_make'] = df_data['matched_make'].str.lower()
    df_data['matched_model'] = df_data['matched_model'].str.lower()
    df_details['make'] = df_details['make'].str.lower()
    df_details['model'] = df_details['model'].str.lower()

    # Merge the dataframes on relevant keys
    merged_df = pd.merge(df_data, df_details, how='left', left_on=['matched_make',
    'matched_model'], right_on=['make', 'model', 'year'])
    [columns_to_fill.values()]

    # Fill missing values in target columns using corresponding source columns
    for target_col, source_col in columns_to_fill.items():
        df_data[target_col] = df_data[target_col].fillna(merged_df[source_col])

    return df_data
```

- Function สำหรับ clean และ group ข้อมูลให้อยู่ในรูปแบบเดียวกัน

```
def group_values(df):
    """
    Clean and preprocess specific columns in the dataframe.

    Parameters:
    df (pd.DataFrame): The dataframe to clean.

    Returns:
    pd.DataFrame: The cleaned dataframe.
    """
    # Clean the 'cylinders' column to keep only numeric values
    df['cylinders'] = df['cylinders'].str.extract(r'(\d+)').astype('Int64')
    df['transmission'] = df['transmission'].str.split(' ').str[0].str.lower()

    # Group the 'fuel' column into broader categories
    fuel_mapping = {
        'gas': 'gasoline',
        'Regular Gasoline': 'gasoline',
        'Premium Gasoline': 'gasoline',
        'Midgrade Gasoline': 'gasoline',
        'diesel': 'diesel',
        'Diesel': 'diesel',
        'electricity': 'electricity',
        'Electricity': 'electricity',
        'hybrid': 'hybrid'
    }
    df['fuel'] = df['fuel'].map(fuel_mapping).fillna(df['fuel'])

    # Group the 'type' column into broader categories
    type_mapping = {
        'sedan': 'car',
        'coupe': 'car',
        'convertible': 'car',
        'hatchback': 'car',
        'wagon': 'car',
        'Midsize Cars': 'car',
        'Compact Cars': 'car',
        'Large Cars': 'car',
        'Subcompact Cars': 'car',
        'Minicompact Cars': 'car',
        'Two Seaters': 'car',
        'SUV': 'suv',
        'Sport Utility Vehicle - 4WD': 'suv',
        'Sport Utility Vehicle - 2WD': 'suv',
        'Small Sport Utility Vehicle 4WD': 'suv',
        'Small Sport Utility Vehicle 2WD': 'suv',
        'Standard Sport Utility Vehicle 4WD': 'suv',
        'Standard Sport Utility Vehicle 2WD': 'suv',
        'Special Purpose Vehicles': 'suv',
        'Special Purpose Vehicle 4WD': 'suv',
        'Special Purpose Vehicle 2WD': 'suv',
        'offroad': 'suv',
        'pickup': 'truck',
        'truck': 'truck',
        'Standard Pickup Trucks 4WD': 'truck',
        'Standard Pickup Trucks 2WD': 'truck',
        'Small Pickup Trucks 4WD': 'truck',
        'Small Pickup Trucks 2WD': 'truck',
        'Small Pickup Trucks': 'truck',
        'Standard Pickup Trucks': 'truck',
        'mini-van': 'van',
        'van': 'van',
        'bus': 'van',
        'Minivan - 2WD': 'van',
        'Minivan - 4WD': 'van',
        'Vans': 'van',
        'Vans, Cargo Type': 'van',
        'Vans, Passenger Type': 'van',
        'Small Station Wagons': 'car',
        'Midsize Station Wagons': 'car'
    }
    df['type'] = df['type'].map(type_mapping).fillna(df['type'])

    return df
```

- Function สำหรับ clean ข้อมูลจากการใช้ function ที่สร้างมาทั้งหมด

```
def clean_data(df_data, df_details):
    """
    Clean the sales and details dataframes by removing rows with missing values
    and resetting the index.

    Parameters:
    df_data (pd.DataFrame): The sales dataframe.
    df_details (pd.DataFrame): The details dataframe.

    Returns:
    pd.DataFrame, pd.DataFrame: The cleaned sales and details dataframes.
    """

    # Select relevant columns
    df_data, df_details = df, df_car_details
    # Replace 'other' values with NaN
    df_data.replace('other', np.nan, inplace=True)

    # Merge the dataframes to get more details
    df_data = get_matched_models(df_data, df_details)
    df_data = df_data[~df_data['matched_make'].isnull() &
~df_data['matched_model'].isnull()]

    # model_dependent_columns = ['cylinders', 'fuel', 'transmission', 'drive',
'size', 'type']
    model_dependent_columns = df_data

    # Fill model dependent values using df_details
    df_data = fill_details(df_data, df_details)
    df_data = group_values(df_data)

    # Fill model dependent values using modes of each model
    df_data = fill_model_deps(df_data, model_dependent_columns)

    df_data = df_data.dropna()

    return df_data
```

- Cleaning df_cleanwfill ด้วย function clean_data

```
df_cleanwfill = clean_data(df, df_car_details)
```

Could not render content for 'application/vnd.jupyter.widget-view+json'
{"model_id": "4e4cdaba149843dfa6c80d11bb3ff41e", "version_major": 2, "version_minor": 0}

Could not render content for 'application/vnd.jupyter.widget-view+json'
{"model_id": "9de547848f8b466a8c4a54605626471e", "version_major": 2, "version_minor": 0}

Matched: 379521 out of 426880 rows

- ลบ row ที่มีค่าที่หายไป (missing values) มากกว่า 1 ค่า

```
# Drop rows that contain at least one NaN value
print(f"Remaining records before fill with stat >> {df.shape[0]}")

# Show result
print(f"Remaining records after fill with stat >> {df_cleanwfill.shape[0]}")
```

Remaining records before fill with stat >> 426880

Remaining records after fill with stat >> 374817

- ลบข้อมูลซ้ำ

```

print(f'Total records (after fill stat) of df >> {df_cleanwfill.shape[0]}')
print(f'Total Duplicated records >> {df_cleanwfill.duplicated().sum()}')

print(f'Non-Duplicated records >> {df_cleanwfill.shape[0] -
df_cleanwfill.duplicated().sum()}')

Total records (after fill stat) of df >> 374817
Total Duplicated records >> 26
Non-Duplicated records >> 374791

```

- เก็บค่าไว้ในตัวแปร df_cleanwfill โดยเก็บเฉพาะแถวที่ไม่ซ้ำและแสดงผลลัพธ์หลังใช้ function

```

df_cleanwfill = df_cleanwfill[~(df_cleanwfill.duplicated())].copy()
df_cleanwfill.info()

```

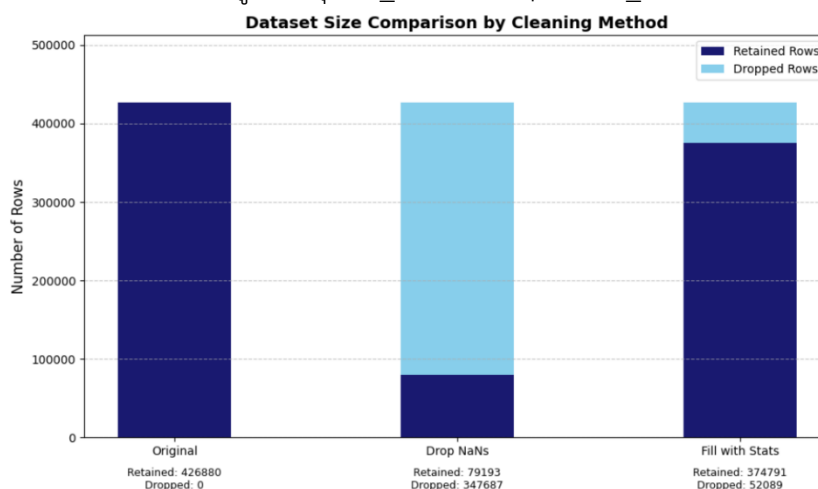
ผลลัพธ์

```

<class 'pandas.core.frame.DataFrame'>
Index: 374791 entries, 27 to 426879
Data columns (total 19 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   region          374791 non-null object
1   price           374791 non-null float64
2   year            374791 non-null float64
3   manufacturer    374791 non-null object
4   model           374791 non-null object
5   condition       374791 non-null object
6   cylinders       374791 non-null Int64
7   fuel            374791 non-null object
8   odometer        374791 non-null float64
9   title_status    374791 non-null object
10  transmission     374791 non-null object
11  drive            374791 non-null object
12  size             374791 non-null object
13  type             374791 non-null object
14  paint_color      374791 non-null object
15  state            374791 non-null object
16  posting_date     374791 non-null datetime64[ns, UTC]
17  matched_make     374791 non-null object
18  matched_model    374791 non-null object
dtypes: Int64(1), datetime64[ns, UTC](1), float64(3), object(14)
memory usage: 57.5+ MB

```

- เปรียบเทียบขนาดข้อมูลของชุด df_cleanwdrop กับ df_cleanwfill



3.2.3 การจัดการข้อมูล (Data Preprocessing)

- โหลดข้อมูลที่ทำกร clean ด้วยการ drop และ fill

```
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

df_cleanwdrop = pd.read_csv('../dataset/cleanedwdrop_sales_data.csv')
df_cleanwfill = pd.read_csv('../dataset/cleanedwfill_sales_data.csv')
```

- แปลงค่าใน column posting_date ให้อยู่ในรูป วันเวลา, price ให้เป็น int และแสดงผลลัพธ์

```
for df in [df_cleanwdrop, df_cleanwfill]:
    df["posting_date"] = pd.to_datetime(df["posting_date"], errors="coerce",
    utc=True)
    df["price"] = df["price"].astype(int)
df_cleanwdrop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 79193 entries, 0 to 79192
Data columns (total 17 columns):
#   Column              Non-Null Count  Dtype
---  -
0   region              79193 non-null object
1   price               79193 non-null int64
2   year               79193 non-null float64
3   manufacturer        79193 non-null object
4   model              79193 non-null object
5   condition           79193 non-null object
6   cylinders           79193 non-null object
7   fuel               79193 non-null object
8   odometer           79193 non-null float64
9   title_status        79193 non-null object
10  transmission         79193 non-null object
11  drive               79193 non-null object
12  size                79193 non-null object
13  type                79193 non-null object
14  paint_color         79193 non-null object
15  state               79193 non-null object
16  posting_date        79193 non-null datetime64[ns, UTC]
dtypes: datetime64[ns, UTC](1), float64(2), int64(1), object(13)
memory usage: 10.3+ MB
```

- ดึงค่า year, month, day จากคอลัมน์ posting_date และคำนวณ car_age

```

# Extract year, month, and day from 'posting_date' and calculate 'car_age' for both datasets
for df in [df_cleanwdrop, df_cleanwfill]:
    df["posting_year"] = pd.to_datetime(df["posting_date"]).dt.year
    df["posting_month"] = pd.to_datetime(df["posting_date"]).dt.month
    df["posting_day"] = pd.to_datetime(df["posting_date"]).dt.day
    df["car_age"] = df["posting_year"] - df["year"]
    df.drop(columns=["posting_date"], inplace=True)

```

- การลบค่าผิดปกติออกจากข้อมูล (Outlier removal)
 - manual ตัดข้อมูลของ price และ odometer ≤ 0 ออก

```

for name, df in [(df_cleanwfill, df_cleanwfill), (df_cleanwdrop, df_cleanwdrop)]:
    print(f'Number of cars that have a car_age <= 0 in {name} >> {len(df[df["car_age"] <= 0])}')
    df = df[df["car_age"] > 0]

    print(f'Number of "parts only" cars in {name} >> {len(df[df["title_status"] == "parts only"])}')
    df = df[df["title_status"] != "parts only"]

    # Update the original variable
    if name == "df_cleanwfill":
        df_cleanwfill = df
    else:
        df_cleanwdrop = df

for name, df in [(df_cleanwdrop, df_cleanwdrop), (df_cleanwfill, df_cleanwfill)]:
    print(f"✅ Max price ({name}):", df["price"].max())
    print(f"✅ Min price ({name}):", df["price"].min())

```

```

Number of cars that have a car_age <= 0 in df_cleanwfill >> 2122
Number of "parts only" cars in df_cleanwfill >> 109
Number of cars that have a car_age <= 0 in df_cleanwdrop >> 180
Number of "parts only" cars in df_cleanwdrop >> 46

```

```

✅ Max price (df_cleanwdrop): 3736928711
✅ Min price (df_cleanwdrop): 0
✅ Max price (df_cleanwfill): 3736928711
✅ Min price (df_cleanwfill): 0

```


- ลอง take log และดูผลลัพธ์ว่าข้อมูล skewed น้อยลงหรือไม่

```

for name, df in [("df_cleanwdrop", df_cleanwdrop), ("df_cleanwfill",
df_cleanwfill)]:
    df = df[(df["price"] > 0) & (df["odometer"] > 0)]
    if name == "df_cleanwdrop":
        df_cleanwdrop = df
    else:
        df_cleanwfill = df

import seaborn as sns

import matplotlib.pyplot as plt
cols = ['price', 'year', 'odometer', 'car_age']

plt.figure(figsize=(20, 8))

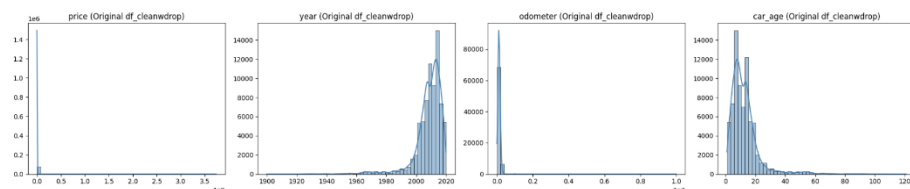
for i, col in enumerate(cols):
    plt.subplot(2, len(cols), i + 1)
    sns.histplot(df_cleanwdrop[col], bins=50, kde=True, color='steelblue',
edgecolor='black')
    plt.title(f'{col} (Original df_cleanwdrop)')
    plt.xlabel("")
    plt.ylabel("")

# Second row: log1p transformed (just an example; customize as needed)
for i, col in enumerate(cols):
    plt.subplot(2, len(cols), i + 1 + len(cols))
    sns.histplot(np.log1p(df_cleanwdrop[col]), bins=50, kde=True,
color='darkorange', edgecolor='black')
    plt.title(f'{col} (Log1p df_cleanwdrop)')
    plt.xlabel("")
    plt.ylabel("")

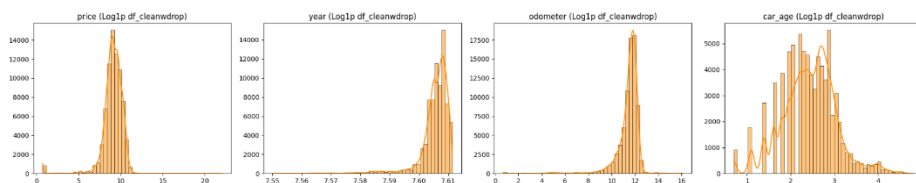
plt.tight_layout()
plt.show()

```

ของข้อมูลจาก df_cleanwdrop ก่อน take log



หลัง take log



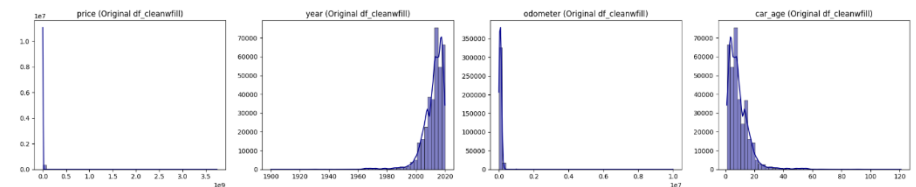
```
plt.figure(figsize=(20, 8))

for i, col in enumerate(cols):
    plt.subplot(2, len(cols), i + 1)
    sns.histplot(df_cleanwfill[col], bins=50, kde=True, color='darkblue',
edgecolor='black')
    plt.title(f'{col} (Original df_cleanwfill)')
    plt.xlabel("")
    plt.ylabel("")

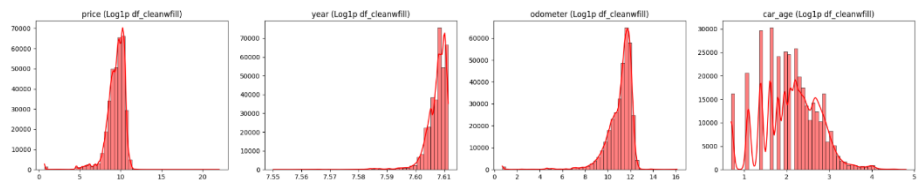
# Second row: log1p transformed (just an example; customize as needed)
for i, col in enumerate(cols):
    plt.subplot(2, len(cols), i + 1 + len(cols))
    sns.histplot(np.log1p(df_cleanwfill[col]), bins=50, kde=True, color='red',
edgecolor='black')
    plt.title(f'{col} (Log1p df_cleanwfill)')
    plt.xlabel("")
    plt.ylabel("")

plt.tight_layout()
plt.show()
```

ของข้อมูลจาก df_cleanwfill ก่อน take log



หลัง take log



○ IQR + log tranformation

```
def clean_and_log_iqr(df, df_name):
    df_log = df.copy()
    cols_to_clean = ['price', 'odometer']

    for col in cols_to_clean:
        df_log[col] = np.log1p(df_log[col]) # Log transform
        Q1 = df_log[col].quantile(0.25)
        Q3 = df_log[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        print(f"[{df_name}] - {col} IQR bounds:")
        print(f"→ lower: {np.expm1(lower_bound):.2f}")
        print(f"→ upper: {np.expm1(upper_bound):.2f}")

    # Filter
    df_log = df_log[(df_log[col] >= lower_bound) & (df_log[col] <= upper_bound)]

    # Inverse transform
    df_log[col] = np.expm1(df_log[col])

    print(f"✅ Final cleaned dataset ({df_name}): {df_log.shape[0]} rows remaining")
    return df_log

# Apply to both DataFrames
df_cleanwdrop_log = clean_and_log_iqr(df_cleanwdrop, "df_cleanwdrop")
df_cleanwfill_log = clean_and_log_iqr(df_cleanwfill, "df_cleanwfill")
```

```
[df_cleanwdrop - price] IQR bounds:
→ lower: 926.58
→ upper: 106,626.66
[df_cleanwdrop - odometer] IQR bounds:
→ lower: 25,596.33
→ upper: 466,150.50
✅ Final cleaned dataset (df_cleanwdrop): 68787 rows remaining
[df_cleanwfill - price] IQR bounds:
→ lower: 928.55
→ upper: 202,599.53
[df_cleanwfill - odometer] IQR bounds:
→ lower: 6,310.06
→ upper: 848,158.78
✅ Final cleaned dataset (df_cleanwfill): 320530 rows remaining
```

```
def apply_iqr_filter(df, df_name, cols_to_filter):
    for col in cols_to_filter:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        print(f"[{df_name} - {col}] IQR bounds: {lower_bound:.2f} to {upper_bound:.2f}")
        before = df.shape[0]
        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
        after = df.shape[0]
        print(f"→ Removed {before - after} rows from '{col}'")

    print(f"✅ Final cleaned dataset ({df_name}): {df.shape[0]} rows remaining")
    return df

# Apply IQR filtering (no log transform) to both datasets
cols_to_filter = ['year', 'car_age']
df_cleanwdrop_log = apply_iqr_filter(df_cleanwdrop_log, "df_cleanwdrop_log",
                                     cols_to_filter)
df_cleanwfill_log = apply_iqr_filter(df_cleanwfill_log, "df_cleanwfill_log",
                                     cols_to_filter)
```

```
[df_cleanwdrop_log - year] IQR bounds: 1991.50 to 2027.50
→ Removed 2676 rows from 'year'
[df_cleanwdrop_log - car_age] IQR bounds: -5.00 to 27.00
→ Removed 389 rows from 'car_age'
✅ Final cleaned dataset (df_cleanwdrop_log): 65722 rows remaining
[df_cleanwfill_log - year] IQR bounds: 1994.50 to 2030.50
→ Removed 7418 rows from 'year'
[df_cleanwfill_log - car_age] IQR bounds: -8.00 to 24.00
→ Removed 1716 rows from 'car_age'
✅ Final cleaned dataset (df_cleanwfill_log): 311396 rows remaining
```

○ Isolation Forest

ของข้อมูลจาก df_cleanwdrop

```

from sklearn.ensemble import IsolationForest

df_cleanwdrop_iso = df_cleanwdrop.copy()
df_cleanwfill_iso = df_cleanwfill.copy()

iso = IsolationForest(contamination=0.02, random_state=42)
# Apply Isolation Forest for anomaly detection on selected columns
columns_to_check = ['price', 'odometer', 'year', 'car_age']

for col in columns_to_check:
    df_cleanwdrop_iso = df_cleanwdrop_iso[iso.fit_predict(df_cleanwdrop_iso[[col]])
    == 1]
    df_cleanwfill_iso = df_cleanwfill_iso[iso.fit_predict(df_cleanwfill_iso[[col]])
    == 1]

cols = ['price', 'year', 'odometer', 'car_age']
datasets = {
    "Original": df_cleanwdrop,
    "Log + IQR": df_cleanwdrop_log,
    "Isolation Forest": df_cleanwdrop_iso
}
colors = ['steelblue', 'darkorange', 'mediumseagreen']

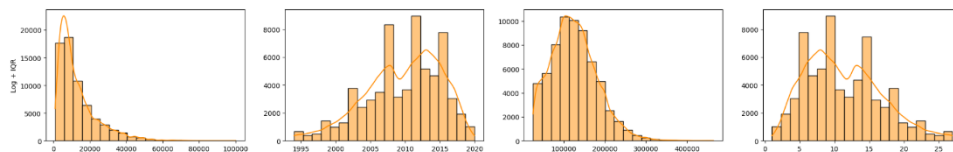
plt.figure(figsize=(20, 10))

for row, (label, df) in enumerate(datasets.items()):
    for i, col in enumerate(cols):
        plt.subplot(len(datasets), len(cols), row * len(cols) + i + 1)
        sns.histplot(df[col], bins=20, kde=True, color=colors[row],
        edgecolor='black')
        if row == 0:
            plt.title(col)
        if i == 0:
            plt.ylabel(label)
        else:
            plt.ylabel("")
            plt.xlabel("")

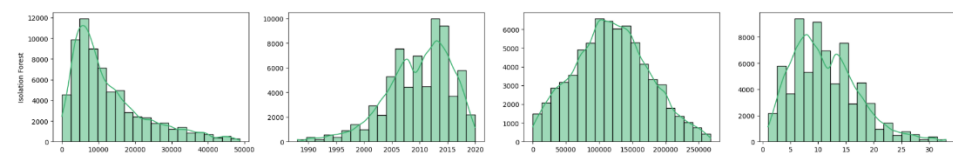
plt.tight_layout()
plt.show()

```

IQR + log transformation



Isolation Forest



ของข้อมูลจาก df_cleanwfill

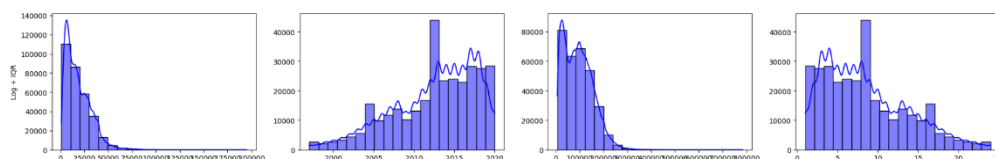
```
cols = ['price', 'year', 'odometer', 'car_age']
datasets = {
    "Original": df_cleanwfill,
    "Log + IQR": df_cleanwfill_log,
    "Isolation Forest": df_cleanwfill_iso
}
colors = ['red', 'blue', 'purple']

plt.figure(figsize=(20, 10))

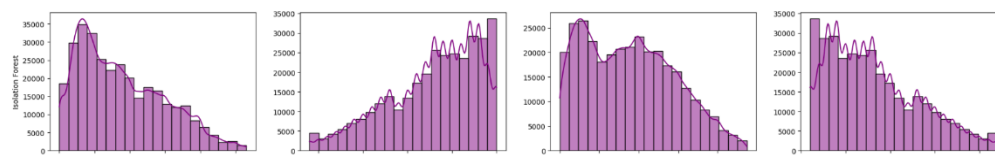
for row, (label, df) in enumerate(datasets.items()):
    for i, col in enumerate(cols):
        plt.subplot(len(datasets), len(cols), row * len(cols) + i + 1)
        sns.histplot(df[col], bins=20, kde=True, color=colors[row],
                     edgecolor='black')
        if row == 0:
            plt.title(col)
        if i == 0:
            plt.ylabel(label)
        else:
            plt.ylabel("")
        plt.xlabel("")

plt.tight_layout()
plt.show()
```

IQR + log tranformation



Isolation forest



- การสร้างฟีเจอร์ (Feature Engineering)

○ import library และดึงข้อมูลเฉพาะ column ที่ต้องการ

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from kneed import KneeLocator
from scipy.stats import boxcox

from sklearn.decomposition import PCA
from mpl_toolkits.mplot3d import Axes3D

# Common feature columns
feature_cols = ['year', 'odometer', 'condition', 'cylinders', 'fuel',
               'title_status',
               'transmission', 'drive', 'type', 'size', 'car_age']

# Dictionary of input and output names
datasets = {
    "feat_df_cleanwfill_log": df_cleanwfill_log,
    "feat_df_cleanwdrop_log": df_cleanwdrop_log,
    "feat_df_cleanwfill_iso": df_cleanwfill_iso,
    "feat_df_cleanwdrop_iso": df_cleanwdrop_iso
}

# Extract selected features
for name, df in datasets.items():
    globals()[name] = df[feature_cols].copy()
```

- normalize ข้อมูลด้วย boxcox และ log

```
def normalize(df):
    # Ensure all values are positive for Box-Cox transformation
    df['odometer'], _ = boxcox(df['odometer'])
    df['car_age'], _ = boxcox(df['car_age'])
    df['year'] = np.log1p(df['year'])
    return df

feat_df_cleanwfill_log = normalize(feat_df_cleanwfill_log)
feat_df_cleanwdrop_log = normalize(feat_df_cleanwdrop_log)
feat_df_cleanwfill_iso = normalize(feat_df_cleanwfill_iso)
feat_df_cleanwdrop_iso = normalize(feat_df_cleanwdrop_iso)
```

- visualize graph หลัง take box-cox และ log

ของข้อมูลจาก df_cleanwdrop

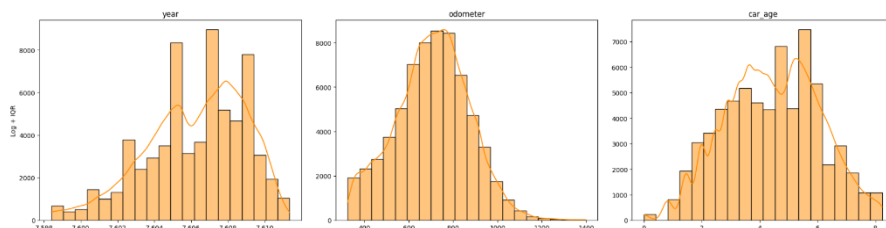
```
cols = ['year', 'odometer', 'car_age']
datasets = {
    "Log + IQR": feat_df_cleanwdrop_log,
    "Isolation Forest": feat_df_cleanwdrop_iso
}
colors = ['darkorange', 'mediumseagreen']

plt.figure(figsize=(20, 10))

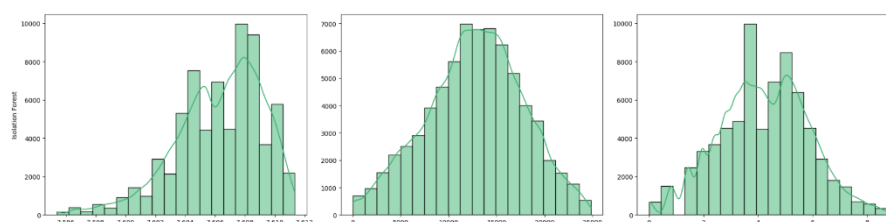
for row, (label, df) in enumerate(datasets.items()):
    for i, col in enumerate(cols):
        plt.subplot(len(datasets), len(cols), row * len(cols) + i + 1)
        sns.histplot(df[col], bins=20, kde=True, color=colors[row])
        if row == 0:
            plt.title(col)
        if i == 0:
            plt.ylabel(label)
        else:
            plt.ylabel("")
        plt.xlabel("")

plt.tight_layout()
plt.show()
```

IQR + log transformation



Isolation forest



ของข้อมูลจาก df_cleanwfill

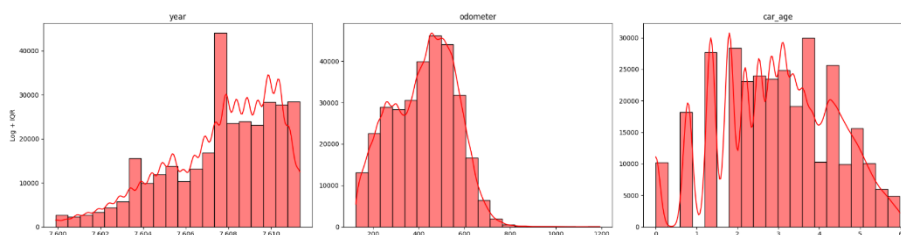
```
cols = ['year', 'odometer', 'car_age']
datasets = {
    "Log + IQR": feat_df_cleanwdrop_log,
    "Isolation Forest": feat_df_cleanwdrop_iso
}
colors = ['darkorange', 'mediumseagreen']

plt.figure(figsize=(20, 10))

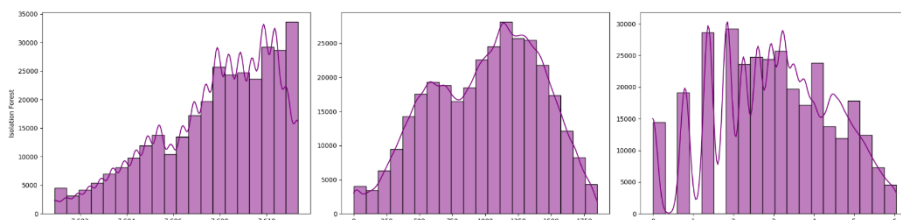
for row, (label, df) in enumerate(datasets.items()):
    for i, col in enumerate(cols):
        plt.subplot(len(datasets), len(cols), row * len(cols) + i + 1)
        sns.histplot(df[col], bins=20, kde=True, color=colors[row])
        if row == 0:
            plt.title(col)
        if i == 0:
            plt.ylabel(label)
        else:
            plt.ylabel("")
        plt.xlabel("")

plt.tight_layout()
plt.show()
```

IQR + log tranformation



Isolation forest



○ การเข้ารหัสข้อมูล (Encoding)

■ manual mapping

```

# Define the mapping
title_status_mapping = {
    'clean': 5,
    'rebuilt': 4,
    'lien': 3,
    'salvage': 2,
    'missing': 1,
    'parts only': 0
}

condition_mapping = {
    'excellent': 5,
    'like new': 4,
    'good': 3,
    'fair': 2,
    'new': 1,
    'salvage': 0
}

cylinder_mapping = {
    '3 cylinders': 3,
    '4 cylinders': 4,
    '5 cylinders': 5,
    '6 cylinders': 6,
    '8 cylinders': 8,
    '10 cylinders': 10,
    '12 cylinders': 12,
    'other': 0
}

# Apply the mapping to the 'title_status' column
for df in [feat_df_cleanwdrop_log, feat_df_cleanwdrop_iso]:
    df['title_status'] =
    df['title_status'].map(title_status_mapping).fillna(5).astype('int64')
    df['condition'] =
    df['condition'].map(condition_mapping).fillna(5).astype('int64')
    df['cylinders'] =
    df['cylinders'].map(cylinder_mapping).fillna(6).astype('int64')

for df in [feat_df_cleanwfill_log, feat_df_cleanwfill_iso]:
    df['title_status'] =
    df['title_status'].map(title_status_mapping).fillna(5).astype('int64')
    df['condition'] =
    df['condition'].map(condition_mapping).fillna(5).astype('int64')

```

■ use Label Encoder

```

# List of DataFrames to process
dataframes = [feat_df_cleanwdrop_log, feat_df_cleanwfill_log,
              feat_df_cleanwdrop_iso, feat_df_cleanwfill_iso]

for df in dataframes:
    # Select categorical columns
    categorical_cols = df.select_dtypes(include='object').columns

    # Apply LabelEncoder to each categorical column
    for col in categorical_cols:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])

```


○ apply PCA

- standard scaler and min-max scaler, eigen value, vector, covariance matrix

```

'''
sc = StandardScaler()
mm = MinMaxScaler()

from sklearn.preprocessing import MinMaxScaler
import numpy as np
import pandas as pd

# Initialize the scaler
mm = MinMaxScaler()

# List of input DataFrames and their names
input_dfs = {
    "cleanwdrop_log": feat_df_cleanwdrop_log,
    "cleanwfill_log": feat_df_cleanwfill_log,
    "cleanwdrop_iso": feat_df_cleanwdrop_iso,
    "cleanwfill_iso": feat_df_cleanwfill_iso,
}

# Output storage
scaled_dfs = {}
centered_dfs = {}
cov_matrices = {}
eigenvalues = {}
eigenvectors = {}

# Processing loop
for name, df in input_dfs.items():
    # Min-Max scaling
    scaled = mm.fit_transform(df)
    scaled_df = pd.DataFrame(scaled, columns=df.columns)
    scaled_dfs[name] = scaled_df

    # Mean centering
    centered = scaled_df - scaled_df.mean()
    centered_dfs[name] = centered

    # Covariance matrix
    cov_matrix = np.cov(centered, rowvar=False)
    cov_matrices[name] = cov_matrix

    # Eigen decomposition
    eigvals, eigvecs = np.linalg.eig(cov_matrix)
    eigenvalues[name] = eigvals
    eigenvectors[name] = eigvecs

```

- check % PVES

```

'''
for name, eigenvalues in eigenvalues.items():
    print(f"\n● PVES for: {name}")
    total_variance_scaler = np.sum(eigenvalues)
    total = 0
    for i, v in enumerate(eigenvalues):
        total += v * 100 / total_variance_scaler
        print(f"PVES {i} : {:.2f} % | Cumulative : {:.2f} %".format(i, v * 100 /
            total_variance_scaler, total))

```

IQR + log tranformation

```

● PVES for: cleanwdrop_log
PVES 0 : 30.67 % | Cumulative : 30.67 %
PVES 1 : 20.84 % | Cumulative : 51.51 %
PVES 2 : 17.06 % | Cumulative : 68.57 %
PVES 3 : 9.25 % | Cumulative : 77.83 %
PVES 4 : 6.92 % | Cumulative : 84.75 %
PVES 5 : 0.09 % | Cumulative : 84.84 %
PVES 6 : 4.05 % | Cumulative : 88.89 %
PVES 7 : 2.32 % | Cumulative : 91.21 %
PVES 8 : 2.52 % | Cumulative : 93.73 %
PVES 9 : 2.88 % | Cumulative : 96.61 %
PVES 10 : 3.39 % | Cumulative : 100.00 %

● PVES for: cleanwfill_log
PVES 0 : 26.95 % | Cumulative : 26.95 %
PVES 1 : 21.26 % | Cumulative : 48.20 %
PVES 2 : 14.12 % | Cumulative : 62.32 %
PVES 3 : 11.28 % | Cumulative : 73.60 %
PVES 4 : 10.05 % | Cumulative : 83.65 %
PVES 5 : 7.29 % | Cumulative : 90.94 %
PVES 6 : 4.06 % | Cumulative : 94.99 %
PVES 7 : 0.23 % | Cumulative : 95.23 %
PVES 8 : 1.97 % | Cumulative : 97.20 %
PVES 9 : 1.31 % | Cumulative : 98.51 %
PVES 10 : 1.49 % | Cumulative : 100.00 %

```

Isolation forest

```

PVES for: cleanwdrop_iso
PVES 0 : 30.06 % | Cumulative : 30.06 %
PVES 1 : 19.86 % | Cumulative : 49.92 %
PVES 2 : 16.49 % | Cumulative : 66.41 %
PVES 3 : 0.11 % | Cumulative : 66.52 %
PVES 4 : 9.45 % | Cumulative : 75.96 %
PVES 5 : 6.78 % | Cumulative : 82.74 %
PVES 6 : 5.13 % | Cumulative : 87.87 %
PVES 7 : 4.00 % | Cumulative : 91.87 %
PVES 8 : 3.35 % | Cumulative : 95.22 %
PVES 9 : 2.35 % | Cumulative : 97.57 %
PVES 10 : 2.43 % | Cumulative : 100.00 %

PVES for: cleanwfill_iso
PVES 0 : 25.64 % | Cumulative : 25.64 %
PVES 1 : 24.40 % | Cumulative : 50.04 %
PVES 2 : 13.21 % | Cumulative : 63.25 %
PVES 3 : 10.60 % | Cumulative : 73.85 %
PVES 4 : 9.24 % | Cumulative : 83.09 %
PVES 5 : 6.81 % | Cumulative : 89.90 %
PVES 6 : 0.22 % | Cumulative : 90.12 %
PVES 7 : 3.82 % | Cumulative : 93.93 %
PVES 8 : 3.17 % | Cumulative : 97.10 %
PVES 9 : 1.51 % | Cumulative : 98.61 %
PVES 10 : 1.39 % | Cumulative : 100.00 %

```

- apply PCA that makes PVES > 90%

```

from sklearn.decomposition import PCA

# Apply PCA with 9 components to log-scaled datasets
pca_log_cleanwdrop = PCA(n_components=8)
X_log_reduced_cleanwdrop =
pca_log_cleanwdrop.fit_transform(scaled_dfs['cleanwdrop_log'])

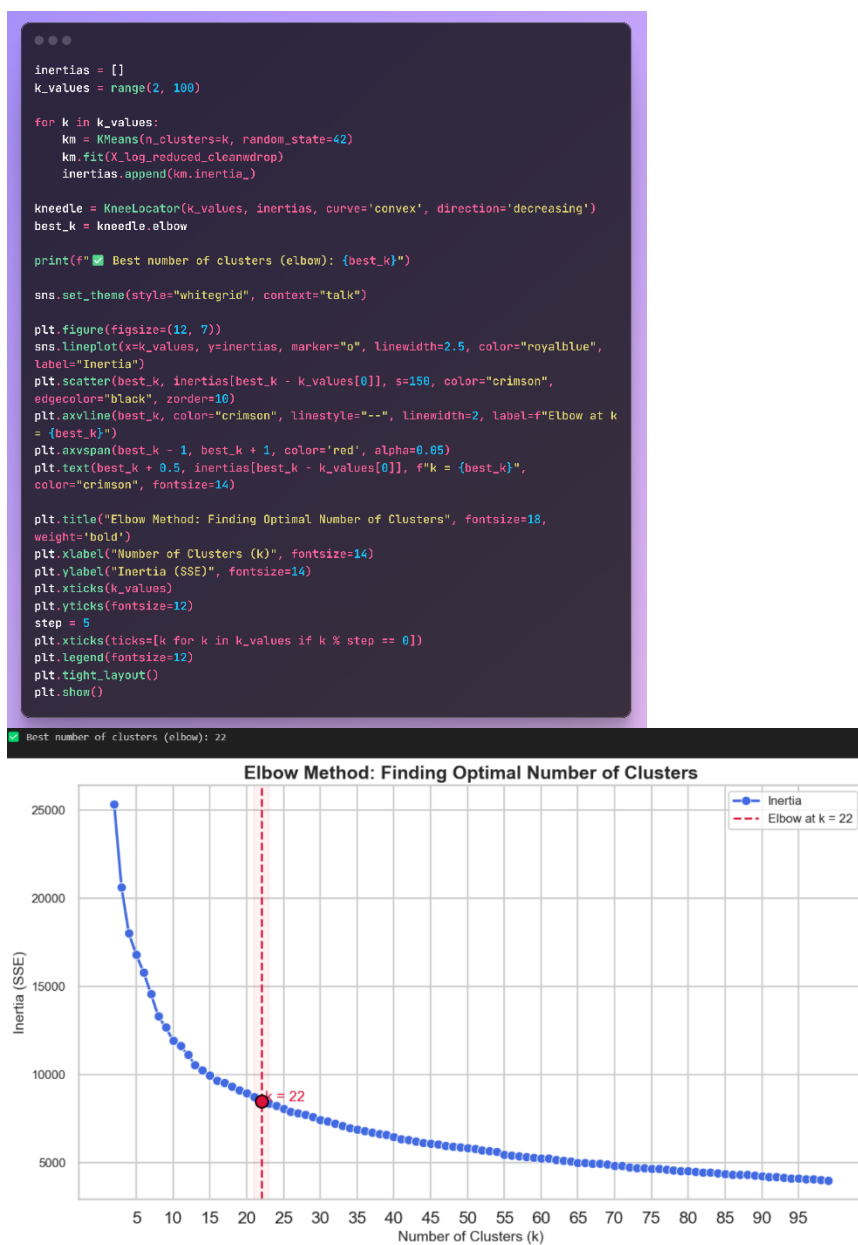
pca_log_cleanwfill = PCA(n_components=6)
X_log_reduced_cleanwfill =
pca_log_cleanwfill.fit_transform(scaled_dfs['cleanwfill_log'])

# Apply PCA with 8 components to ISO-scaled datasets
pca_iso_cleanwdrop = PCA(n_components=7)
X_iso_reduced_cleanwdrop =
pca_iso_cleanwdrop.fit_transform(scaled_dfs['cleanwdrop_iso'])

pca_iso_cleanwfill = PCA(n_components=7)
X_iso_reduced_cleanwfill =
pca_iso_cleanwfill.fit_transform(scaled_dfs['cleanwfill_iso'])

```

- apply elbow method เพื่อหาจำนวน cluster ของแต่ละ dataset



```

inertias = []
k_values = range(2, 100)

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_iss_reduced_cleanwdrop)
    inertias.append(km.inertia_)

kneedle = KneedleLocator(k_values, inertias, curve='convex', direction='decreasing')
best_k = kneedle.elbow

print(f"✅ Best number of clusters (elbow): {best_k}")

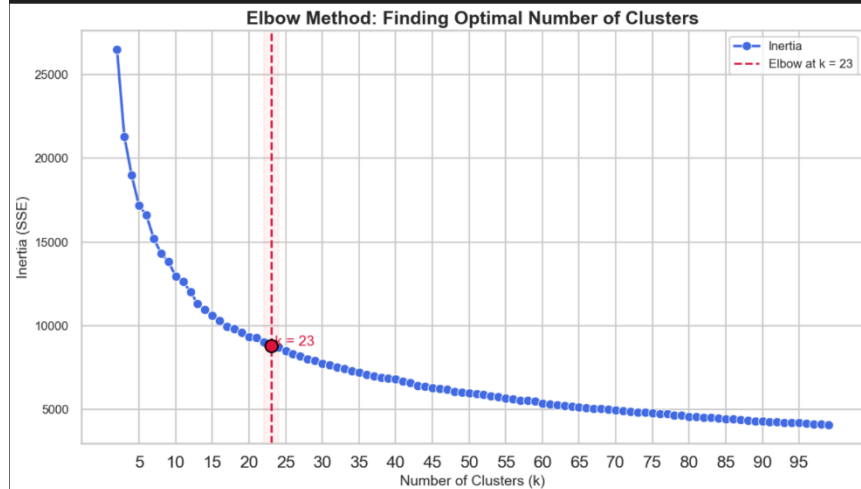
sns.set_theme(style="whitegrid", context="talk")

plt.figure(figsize=(12, 7))
sns.lineplot(x=k_values, y=inertias, marker="o", linewidth=2.5, color="royalblue",
             label="Inertia")
plt.scatter(best_k, inertias[best_k - k_values[0]], s=150, color="crimson",
            edgecolor="black", zorder=10)
plt.axvline(best_k, color="crimson", linestyle="--", linewidth=2, label=f"Elbow at k = {best_k}")
plt.axvspan(best_k - 1, best_k + 1, color='red', alpha=0.05)
plt.text(best_k + 0.5, inertias[best_k - k_values[0]], f"k = {best_k}",
         color="crimson", fontsize=14)

plt.title("Elbow Method: Finding Optimal Number of Clusters", fontsize=18,
         weight='bold')
plt.xlabel("Number of Clusters (k)", fontsize=14)
plt.ylabel("Inertia (SSE)", fontsize=14)
plt.xticks(k_values)
plt.yticks(fontsize=12)
step = 5
plt.xticks(ticks=[k for k in k_values if k % step == 0])
plt.legend(fontsize=12)
plt.tight_layout()
plt.show()

```

✅ Best number of clusters (elbow): 23



```

inertias = []
k_values = range(2, 100)

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_log_reduced_cleanwfill)
    inertias.append(km.inertia_)

kneedle = KneedleLocator(k_values, inertias, curve='convex', direction='decreasing')
best_k = kneedle.elbow

print(f"👍 Best number of clusters (elbow): {best_k}")

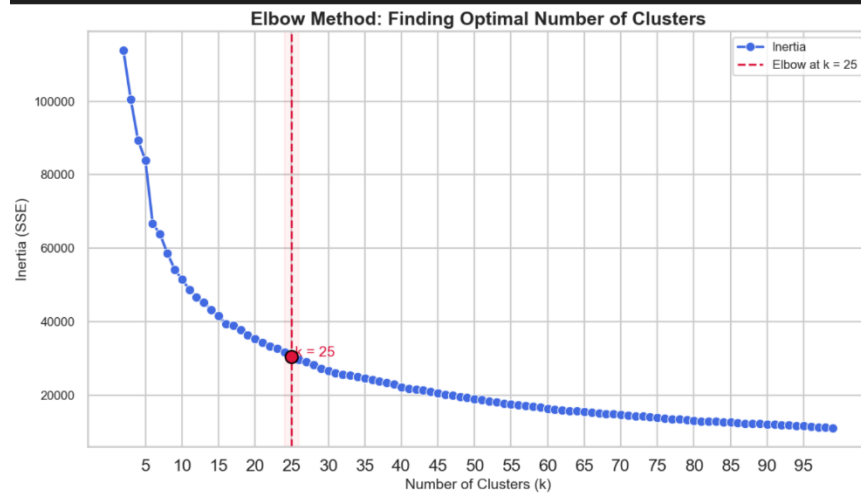
sns.set_theme(style="whitegrid", context="talk")

plt.figure(figsize=(12, 7))
sns.lineplot(x=k_values, y=inertias, marker="o", linewidth=2.5, color="royalblue",
             label="Inertia")
plt.scatter(best_k, inertias[best_k - k_values[0]], s=150, color="crimson",
            edgecolor="black", zorder=10)
plt.axvline(best_k, color="crimson", linestyle="--", linewidth=2, label=f"Elbow at k = {best_k}")
plt.axvspan(best_k - 1, best_k + 1, color='red', alpha=0.05)
plt.text(best_k + 0.5, inertias[best_k - k_values[0]], f"k = {best_k}",
         color="crimson", fontsize=14)

plt.title("Elbow Method: Finding Optimal Number of Clusters", fontsize=18,
         weight='bold')
plt.xlabel("Number of Clusters (k)", fontsize=14)
plt.ylabel("Inertia (SSE)", fontsize=14)
plt.xticks(k_values)
plt.yticks(fontsize=12)
step = 5
plt.xticks(ticks=[k for k in k_values if k % step == 0])
plt.legend(fontsize=12)
plt.tight_layout()
plt.show()

```

👍 Best number of clusters (elbow): 25



```

inertias = []
k_values = range(2, 100)

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_iso_reduced_cleanfill)
    inertias.append(km.inertia_)

kneedle = Kneelocator(k_values, inertias, curve='convex', direction='decreasing')
best_k = kneedle.elbow

print(f"👉 Best number of clusters (elbow): {best_k}")

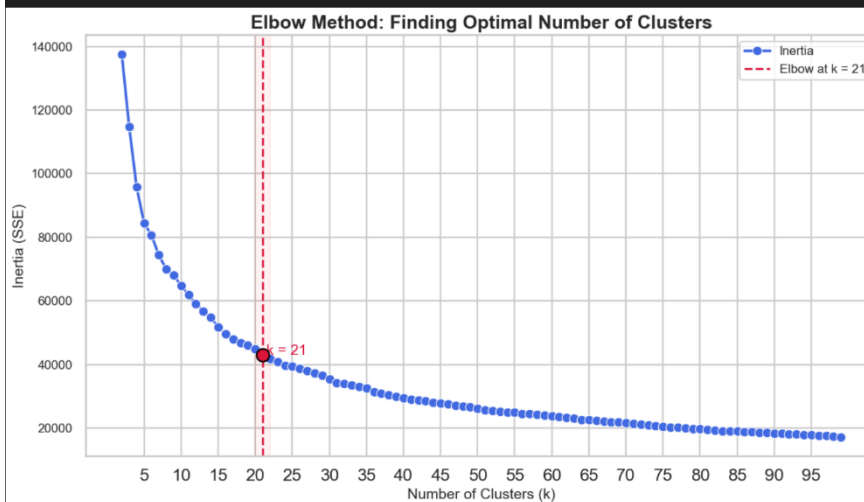
sns.set_theme(style="whitegrid", context="talk")

plt.figure(figsize=(12, 7))
sns.lineplot(x=k_values, y=inertias, marker="o", linewidth=2.5, color="royalblue",
             label="Inertia")
plt.scatter(best_k, inertias[best_k - k_values[0]], s=150, color="crimson",
            edgecolor="black", zorder=10)
plt.axvline(best_k, color="crimson", linestyle="--", linewidth=2, label=f"Elbow at k = {best_k}")
plt.axvspan(best_k - 1, best_k + 1, color='red', alpha=0.05)
plt.text(best_k + 0.5, inertias[best_k - k_values[0]], f"k = {best_k}",
         color="crimson", fontsize=14)

plt.title("Elbow Method: Finding Optimal Number of Clusters", fontsize=18,
         weight='bold')
plt.xlabel("Number of Clusters (k)", fontsize=14)
plt.ylabel("Inertia (SSE)", fontsize=14)
plt.xticks(k_values)
plt.yticks(fontsize=12)
step = 5
plt.xticks(ticks=[k for k in k_values if k % step == 0])
plt.legend(fontsize=12)
plt.tight_layout()
plt.show()

```

👉 Best number of clusters (elbow): 21



■ visualize PCA cluster

```

from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import seaborn as sns
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def visualize_pca_clusters(X_reduced, df_with_labels, title_prefix=''):
    # PCA transformation
    pca_2d = PCA(n_components=2)
    pca_3d = PCA(n_components=3)
    pca_2d_result = pca_2d.fit_transform(X_reduced)
    pca_3d_result = pca_3d.fit_transform(X_reduced)

    # Set theme
    sns.set_theme(style="whitegrid")
    fig = plt.figure(figsize=(16, 7))

    # ----- 2D PCA -----
    ax1 = fig.add_subplot(1, 2, 1)
    sns.scatterplot(
        x=pca_2d_result[:, 0], y=pca_2d_result[:, 1],
        hue=df_with_labels['car_segment'],
        palette='tab20', s=40, ax=ax1, legend='full'
    )
    ax1.set_title(f'{title_prefix} 2D PCA of Car Segments', fontsize=14)
    ax1.set_xlabel('PCA 1', fontsize=12)
    ax1.set_ylabel('PCA 2', fontsize=12)
    ax1.legend(title='car_segment', bbox_to_anchor=(1.05, 1), loc='upper left')

    # ----- 3D PCA -----
    ax2 = fig.add_subplot(1, 2, 2, projection='3d')
    scatter = ax2.scatter(
        pca_3d_result[:, 0], pca_3d_result[:, 1], pca_3d_result[:, 2],
        c=df_with_labels['car_segment'], cmap='tab20', s=30, alpha=0.8
    )
    ax2.set_title(f'{title_prefix} 3D PCA of Car Segments', fontsize=14)
    ax2.set_xlabel('PCA 1', fontsize=10)
    ax2.set_ylabel('PCA 2', fontsize=10)
    ax2.set_zlabel('PCA 3', fontsize=10)
    ax2.view_init(elev=20, azim=135)
    ax2.set_box_aspect([1, 1, 1])

    plt.tight_layout()
    plt.show()

# -----
# Apply clustering & visualize for all 4 sets
# -----

kmeans = KMeans(n_clusters=21, random_state=42)

# Log - Drop
df_cleanwdrop_log['car_segment'] = kmeans.fit_predict(X_log_reduced_cleanwdrop)
visualize_pca_clusters(X_log_reduced_cleanwdrop, df_cleanwdrop_log,
title_prefix='Log Drop')

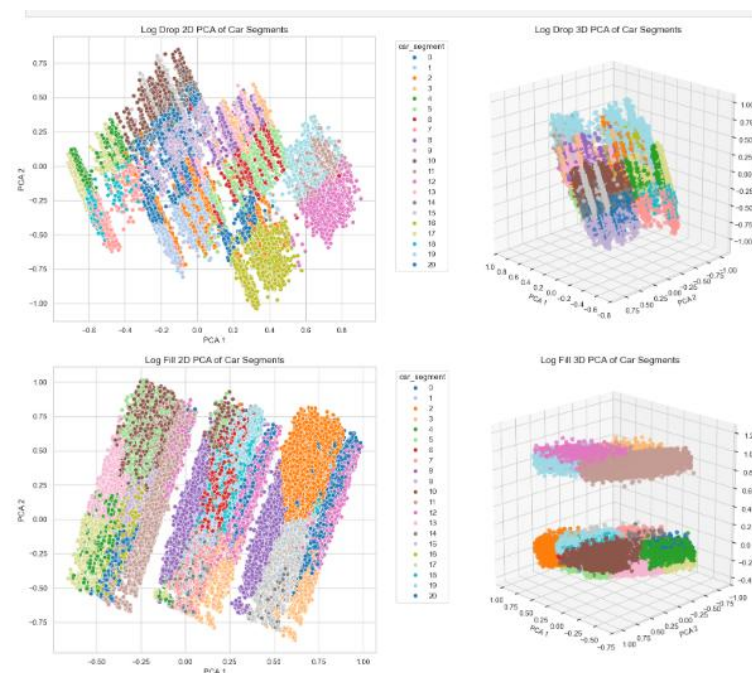
# Log - Fill
df_cleanwfill_log['car_segment'] = kmeans.fit_predict(X_log_reduced_cleanwfill)
visualize_pca_clusters(X_log_reduced_cleanwfill, df_cleanwfill_log,
title_prefix='Log Fill')

# ISO - Drop
df_cleanwdrop_iso['car_segment'] = kmeans.fit_predict(X_iso_reduced_cleanwdrop)
visualize_pca_clusters(X_iso_reduced_cleanwdrop, df_cleanwdrop_iso,
title_prefix='ISO Drop')

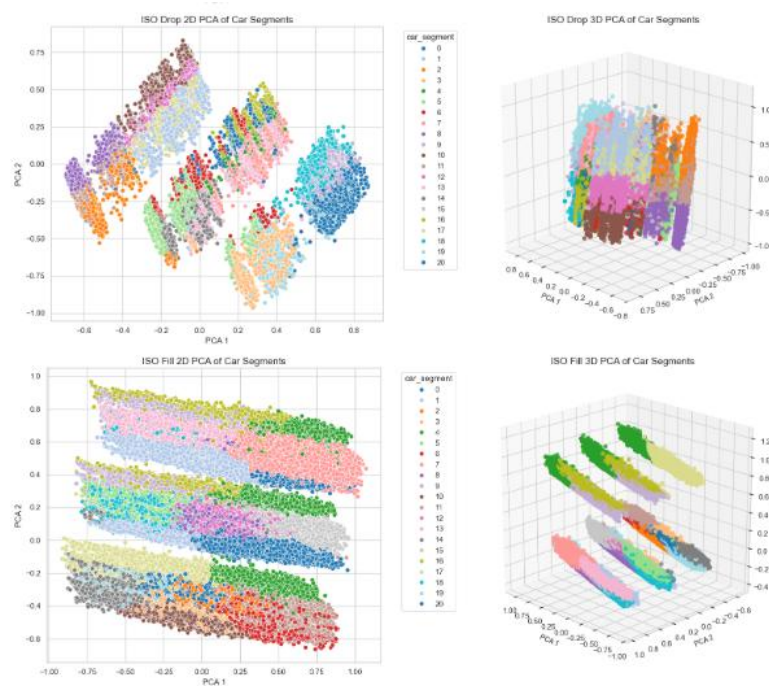
# ISO - Fill
df_cleanwfill_iso['car_segment'] = kmeans.fit_predict(X_iso_reduced_cleanwfill)
visualize_pca_clusters(X_iso_reduced_cleanwfill, df_cleanwfill_iso,
title_prefix='ISO Fill')

```

IQR + log transformation



Isolation Forest



■ ทำการ Encoding

```
df_cleanwdrop_log_encode = df_cleanwdrop_log.copy()
df_cleanwfill_log_encode = df_cleanwfill_log.copy()
df_cleanwdrop_iso_encode = df_cleanwdrop_iso.copy()
df_cleanwfill_iso_encode = df_cleanwfill_iso.copy()

# Define the mappings
title_status_mapping = {
    'clean': 5, 'rebuilt': 4, 'lien': 3,
    'salvage': 2, 'missing': 1, 'parts only': 0
}

condition_mapping = {
    'excellent': 5, 'like new': 4, 'good': 3,
    'fair': 2, 'new': 1, 'salvage': 0
}

cylinder_mapping = {
    '3 cylinders': 3, '4 cylinders': 4, '5 cylinders': 5,
    '6 cylinders': 6, '8 cylinders': 8,
    '10 cylinders': 10, '12 cylinders': 12,
    'other': 0
}

# List of DataFrames to apply the mappings
encoded_dfs = [
    df_cleanwdrop_log_encode,
    df_cleanwdrop_iso_encode
]

encoded_dfsfill = [
    df_cleanwfill_iso_encode,
    df_cleanwfill_log_encode
]

# Apply mappings to each DataFrame
for df in encoded_dfs:
    df['title_status'] = df['title_status'].map(title_status_mapping).fillna(5)
    df['condition'] = df['condition'].map(condition_mapping).fillna(5)
    df['cylinders'] = df['cylinders'].map(cylinder_mapping).fillna(6)

for df in encoded_dfsfill:
    df['title_status'] = df['title_status'].map(title_status_mapping).fillna(5)
    df['condition'] = df['condition'].map(condition_mapping).fillna(5)
```

```
categorical_cols = df_cleanwdrop_log_encode.select_dtypes(include='object').columns

for col in categorical_cols:
    le = LabelEncoder()
    df_cleanwdrop_log_encode[col] = le.fit_transform(df_cleanwdrop_log_encode[col].astype(str))

df_cleanwdrop_log_encode.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 65722 entries, 0 to 79191
Data columns (total 21 columns):
#   Column          Non-Null Count  Dtype
---  -
0   region           65722 non-null  int64
1   price            65722 non-null  float64
2   year             65722 non-null  float64
3   manufacturer     65722 non-null  int64
4   model            65722 non-null  int64
5   condition        65722 non-null  int64
6   cylinders        65722 non-null  int64
7   fuel             65722 non-null  int64
8   odometer         65722 non-null  float64
9   title_status     65722 non-null  int64
10  transmission     65722 non-null  int64
11  drive            65722 non-null  int64
12  size             65722 non-null  int64
13  type             65722 non-null  int64
14  paint_color      65722 non-null  int64
15  state            65722 non-null  int64
16  posting_year     65722 non-null  int32
17  posting_month    65722 non-null  int32
18  posting_day      65722 non-null  int32
19  car_age          65722 non-null  float64
20  car_segment      65722 non-null  int32
dtypes: float64(4), int32(4), int64(13)
memory usage: 10.0 MB
```

```

categorical_cols = df_cleanwfill_log_encode.select_dtypes(include='object').columns

for col in categorical_cols:
    le = LabelEncoder()
    df_cleanwfill_log_encode[col] =
    le.fit_transform(df_cleanwfill_log_encode[col].astype(str))

df_cleanwfill_log_encode.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 311396 entries, 0 to 374790
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   region                 311396 non-null  int64
1   price                  311396 non-null  float64
2   year                   311396 non-null  float64
3   manufacturer            311396 non-null  int64
4   model                  311396 non-null  int64
5   condition               311396 non-null  int64
6   cylinders               311396 non-null  int64
7   fuel                   311396 non-null  int64
8   odometer               311396 non-null  float64
9   title_status           311396 non-null  int64
10  transmission           311396 non-null  int64
11  drive                  311396 non-null  int64
12  size                   311396 non-null  int64
13  type                   311396 non-null  int64
14  paint_color            311396 non-null  int64
15  state                  311396 non-null  int64
16  matched_make           311396 non-null  int64
17  matched_model          311396 non-null  int64
18  posting_year           311396 non-null  int32
19  posting_month          311396 non-null  int32
20  posting_day            311396 non-null  int32
21  car_age                311396 non-null  float64
22  car_segment            311396 non-null  int32
dtypes: float64(4), int32(4), int64(15)
memory usage: 52.3 MB

```

```

categorical_cols = df_cleanwdrop_iso_encode.select_dtypes(include='object').columns

for col in categorical_cols:
    le = LabelEncoder()
    df_cleanwdrop_iso_encode[col] =
    le.fit_transform(df_cleanwdrop_iso_encode[col].astype(str))

df_cleanwdrop_iso_encode.info()

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 69694 entries, 0 to 79191
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   region                 69694 non-null  int64
1   price                  69694 non-null  int64
2   year                   69694 non-null  float64
3   manufacturer            69694 non-null  int64
4   model                  69694 non-null  int64
5   condition               69694 non-null  int64
6   cylinders               69694 non-null  int64
7   fuel                   69694 non-null  int64
8   odometer               69694 non-null  float64
9   title_status           69694 non-null  int64
10  transmission           69694 non-null  int64
11  drive                  69694 non-null  int64
12  size                   69694 non-null  int64
13  type                   69694 non-null  int64
14  paint_color            69694 non-null  int64
15  state                  69694 non-null  int64
16  posting_year           69694 non-null  int32
17  posting_month          69694 non-null  int32
18  posting_day            69694 non-null  int32
19  car_age                69694 non-null  float64
20  car_segment            69694 non-null  int32
dtypes: float64(3), int32(4), int64(14)
memory usage: 10.6 MB

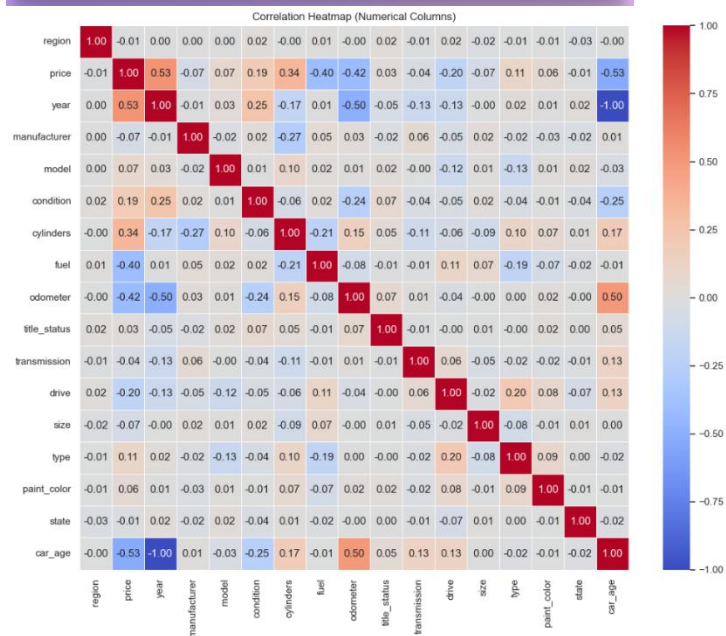
```

- ทำ heatmap

```
# เลือกเฉพาะคอลัมน์ตัวเลข
numerical_cols = df_cleanwdrop_log_encode.select_dtypes(include=['float64',
'int64'])

# สร้าง correlation matrix
corr = numerical_cols.corr()

# Plot heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap (Numerical Columns)")
plt.tight_layout()
plt.show()
```



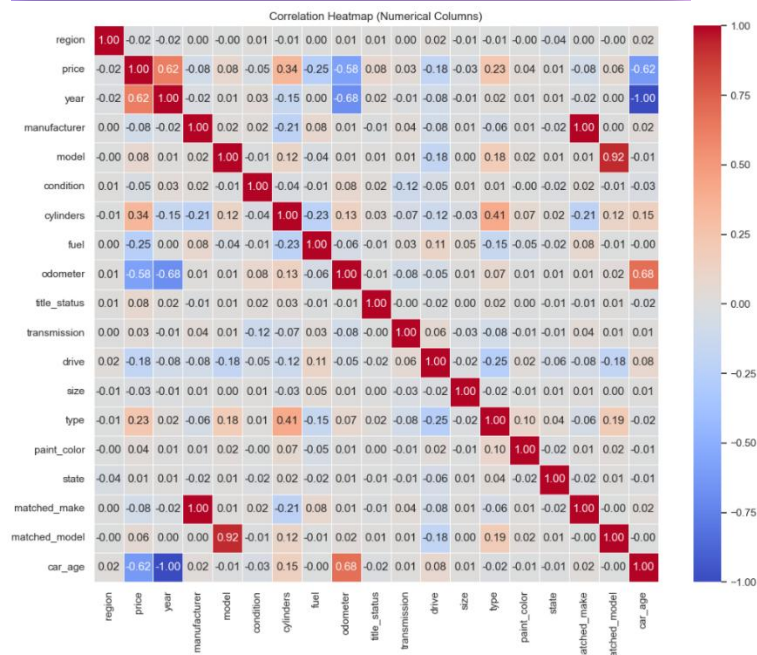
```

# เลือกเฉพาะคอลัมน์ตัวเลข
numerical_cols = df_cleanwfill_log_encode.select_dtypes(include=['float64',
'int64'])

# สร้าง correlation matrix
corr = numerical_cols.corr()

# Plot heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap (Numerical Columns)")
plt.tight_layout()
plt.show()

```



บทที่ 4

วิธีการสร้างแบบจำลองและผลลัพธ์

4.1 การนำเข้าไลบรารี

```
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import cross_val_predict, KFold
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

4.2 การฝึกโมเดล

4.2.1 การฝึกโมเดลแบบไม่มีการปรับแต่งพารามิเตอร์ (Training without tuning)

- ลบ column และแปลงค่าเป็น integer

```
trainwdrop_log.drop(columns=['year'], inplace=True)
trainwdrop_iso.drop(columns=['year'], inplace=True)

trainwfill_log.drop(columns=['year', 'manufacturer', 'model'], inplace=True)
trainwfill_iso.drop(columns=['year', 'manufacturer', 'model'], inplace=True)
#odometer can car_age into int
trainwdrop_log['car_age'] = trainwdrop_log['car_age'].astype(int)
trainwdrop_iso['car_age'] = trainwdrop_iso['car_age'].astype(int)
trainwfill_log['car_age'] = trainwfill_log['car_age'].astype(int)
trainwfill_iso['car_age'] = trainwfill_iso['car_age'].astype(int)

trainwdrop_log['odometer'] = trainwdrop_log['odometer'].astype(int)
trainwdrop_iso['odometer'] = trainwdrop_iso['odometer'].astype(int)
trainwfill_log['odometer'] = trainwfill_log['odometer'].astype(int)
trainwfill_iso['odometer'] = trainwfill_iso['odometer'].astype(int)
```

- set model และ dataset และสร้าง function สำหรับ evaluate

```
models = {
    "LinearRegression": LinearRegression(),
    "RidgeRegression": Ridge(alpha=1.0),
    "RandomForest": RandomForestRegressor(n_jobs=-1, random_state=42),
    "XGBoost": XGBRegressor(n_jobs=-1, random_state=42, verbosity=0,
tree_method='gpu_hist')
}
datasets = {
    "Drop_Log": trainwdrop_log.copy(),
    "Drop_Iso": trainwdrop_iso.copy(),
    "Fill_Log": trainwfill_log.copy(),
    "Fill_Iso": trainwfill_iso.copy()
}
def evaluate(y, y_pred):
    r2 = r2_score(y, y_pred)
    rmse = np.sqrt(mean_squared_error(y, y_pred))
    mae = mean_absolute_error(y, y_pred)
    pmse = (mae / y.mean()) * 100
    return r2, rmse, mae, pmse
```

- ทำ cross validation และ k-fold

```

from tqdm.notebook import tqdm
import pandas as pd
from sklearn.model_selection import KFold, cross_val_predict

kf = KFold(n_splits=5, shuffle=True, random_state=42)
results_no_hyper_tune = []

# Total iterations = number of datasets * number of models
total_iters = len(datasets) * len(models)

# Create progress bar
pbar = tqdm(total=total_iters, desc="Evaluating Models", leave=True)

# --- LOOP THROUGH DATASETS & MODELS ---
for name, df in datasets.items():
    X = df.drop(columns=["price"])
    y = df["price"]

    for model_name, model in models.items():
        y_pred = cross_val_predict(model, X, y, cv=kf, n_jobs=-1)
        r2, rmse, mae, pmae = evaluate(y, y_pred)

        results_no_hyper_tune.append({
            "Dataset": name,
            "Model": model_name,
            "R²": round(r2, 4),
            "RMSE": round(rmse, 2),
            "MAE": round(mae, 2),
            "PMAE (%)": round(pmae, 2)
        })

    # Update progress
    pbar.update(1)

pbar.close()

```

- ทำการ Train

```

from tqdm.notebook import tqdm
import pandas as pd
from sklearn.model_selection import KFold, cross_val_predict

kf = KFold(n_splits=5, shuffle=True, random_state=42)
results_no_hyper_tune = []

# Total iterations = number of datasets * number of models
total_iters = len(datasets) * len(models)

# Create progress bar
pbar = tqdm(total=total_iters, desc="Evaluating Models", leave=True)

# --- LOOP THROUGH DATASETS & MODELS ---
for name, df in datasets.items():
    X = df.drop(columns=["price"])
    y = df["price"]

    for model_name, model in models.items():
        y_pred = cross_val_predict(model, X, y, cv=kf, n_jobs=-1)
        r2, rmse, mae, pmae = evaluate(y, y_pred)

        results_no_hyper_tune.append({
            "Dataset": name,
            "Model": model_name,
            "R²": round(r2, 4),
            "RMSE": round(rmse, 2),
            "MAE": round(mae, 2),
            "PMAE (%)": round(pmae, 2)
        })

    # Update progress
    pbar.update(1)

pbar.close()

```

	Dataset	Model	R ²	RMSE	MAE	PMAE (%)
0	Drop_Log	LinearRegression	0.6535	6,384.77	4,611.21	35.27
1	Drop_Log	RidgeRegression	0.6535	6,384.77	4,611.21	35.27
2	Drop_Log	RandomForest	0.8916	3,571.32	2,001.97	15.31
3	Drop_Log	XGBoost	0.8920	3,564.99	2,216.08	16.95
4	Drop_Iso	LinearRegression	0.5603	6,691.75	4,818.05	37.80
5	Drop_Iso	RidgeRegression	0.5603	6,691.75	4,818.04	37.80
6	Drop_Iso	RandomForest	0.8799	3,497.22	2,038.81	15.99
7	Drop_Iso	XGBoost	0.8704	3,633.03	2,332.95	18.30
8	Fill_Log	LinearRegression	0.6900	7,444.66	5,264.40	28.39
9	Fill_Log	RidgeRegression	0.6900	7,444.66	5,264.40	28.39
10	Fill_Log	RandomForest	0.9248	3,666.07	1,735.70	9.36
11	Fill_Log	XGBoost	0.8931	4,372.10	2,639.08	14.23
12	Fill_Iso	LinearRegression	0.6361	7,307.91	5,252.39	29.65
13	Fill_Iso	RidgeRegression	0.6361	7,307.91	5,252.39	29.65
14	Fill_Iso	RandomForest	0.8903	4,011.87	1,999.36	11.29
15	Fill_Iso	XGBoost	0.8307	4,984.69	3,024.49	17.07

- ทำ heatmap เพื่อดูประสิทธิภาพจากค่า R² ของแต่ละ model

```

import matplotlib.pyplot as plt
import seaborn as sns
# Convert the results List to a DataFrame
results_df = pd.DataFrame(results_no_hyper_tune)

# Create a pivot table for the heatmap
heatmap_data = results_df.pivot(index="Model", columns="Dataset", values="R2")

# Calculate the average R2 and sort by it
heatmap_data["Avg R2"] = heatmap_data.mean(axis=1)
heatmap_data.sort_values("Avg R2", ascending=False, inplace=True)
heatmap_data.drop(columns="Avg R2", inplace=True)

# Red theme color map
plt.figure(figsize=(10, 6))
sns.heatmap(
    heatmap_data,
    annot=True,
    fmt=".3f",
    cmap="Reds",
    linewidths=0.5,
    linecolor='white',
    cbar_kws={"label": "R2 Score"}
)

plt.title("Model Performance by R2 (Sorted)", fontsize=16)
plt.xlabel("Dataset", fontsize=12)
plt.ylabel("Model", fontsize=12)
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)
plt.show()

```



4.2.2 หาความสำคัญของข้อมูล

- หา Feature importance ของ fill log

```

from sklearn.base import clone

# Prepare data
X_Fill = datasets["Fill_Log"].drop(columns=["price"])
y_Fill = datasets["Fill_Log"]["price"]

X_Drop = datasets["Drop_Log"].drop(columns=["price"])
y_Drop = datasets["Drop_Log"]["price"]

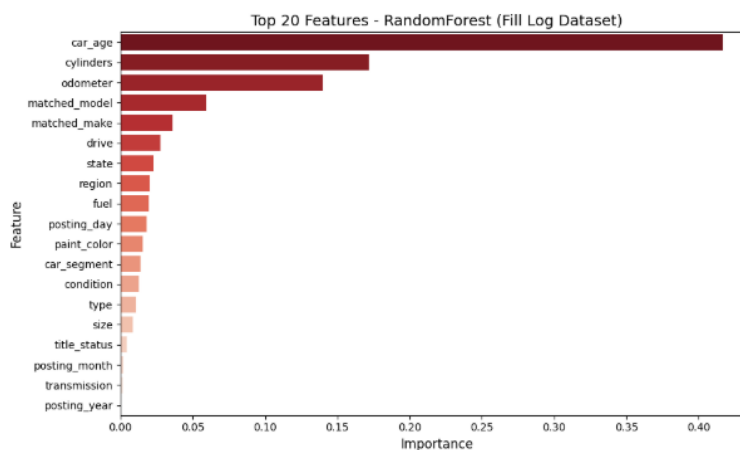
# Fit model
model = clone(models["RandomForest"]).fit(X, y)

# Extract feature importances and select top 20
importances_fill = pd.Series(model.feature_importances_,
                              index=X_Fill.columns).sort_values(ascending=False).head(20)

# Convert to DataFrame for seaborn
top_features_df = importances_fill.reset_index()
top_features_df.columns = ['Feature', 'Importance']

# Plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_features_df,
    x='Importance',
    y='Feature',
    hue='Feature',
    palette='Reds_r',
    dodge=False,
    legend=False # hide redundant legend
)
plt.title("Top 20 Features - RandomForest (Fill Log Dataset)", fontsize=14)
plt.xlabel("Importance", fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.show()

```



- หา Feature importance ของ Drop log

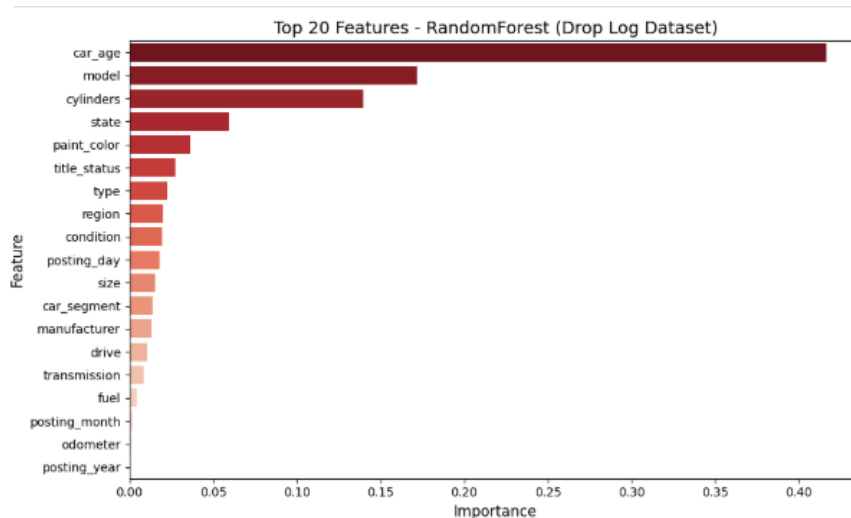
```

# Extract feature importances and select top 20
importances_drop = pd.Series(model.feature_importances_,
                              index=X_Drop.columns).sort_values(ascending=False).head(20)

# Convert to DataFrame for seaborn
top_features_df = importances_drop.reset_index()
top_features_df.columns = ['Feature', 'Importance']

# Plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_features_df,
    x='Importance',
    y='Feature',
    hue='Feature',
    palette='Reds_r',
    dodge=False,
    legend=False # hide redundant legend
)
plt.title("Top 20 Features - RandomForest (Drop Log Dataset)", fontsize=14)
plt.xlabel("Importance", fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.show()

```



- เลือก column หลังทำ feature importance

```

select_feature_fill = importances_fill[:18].index
select_feature_fill
select_feature_drop = importances_drop[:18].index
select_feature_drop

```

4.2.3 การฝึกโมเดลแบบมีการปรับแต่งพารามิเตอร์และเลือกฟีเจอร์ (Training with tuning & Feature selection)

- ทำ Hyperparameter Tuning ด้วย Optuna พร้อมกับ Early Stopping

```

datasets = {
    "Drop_Log": datasets["Drop_Log"][select_feature_drop.tolist() +
    ['price']].copy(),
    "Drop_Iso": datasets["Drop_Iso"][select_feature_drop.tolist() +
    ['price']].copy(),
    "Fill_Log": datasets["Fill_Log"][select_feature_fill.tolist() +
    ['price']].copy(),
    "Fill_Iso": datasets["Fill_Iso"][select_feature_fill.tolist() +
    ['price']].copy()
}

from tqdm.notebook import tqdm
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.model_selection import cross_val_predict, KFold
import numpy as np
import pandas as pd
import optuna

results_tuned = []
best_params = {}

# --- Early stopping callback ---
class EarlyStoppingCallback:
    def __init__(self, patience):
        self.patience = patience
        self.best_value = float('inf')
        self.counter = 0

    def __call__(self, study, trial):
        if study.best_value < self.best_value:
            self.best_value = study.best_value
            self.counter = 0
        else:
            self.counter += 1
        if self.counter >= self.patience:
            print(f"🛑 Early stopping triggered after {self.patience} trials.")
            study.stop()

```

- สร้าง function สำหรับ tuning

```

import joblib
import os

# --- Main tuning loop ---
def run_tuning(dataset_name, model_name, datasets, n_trials=None, patience=None):
    X = datasets[dataset_name].drop(columns=["price"])
    y = datasets[dataset_name]["price"]

    # Check if the model has already been trained on this dataset
    existing_result_index = next(
        (i for i, result in enumerate(results_tuned)
         if result["Dataset"] == dataset_name and
         result["Model"].startswith(model_name)),
        None
    )

    # Set default trial/patience if not specified
    if n_trials is None:
        if model_name == "XGBoost":
            n_trials = 10
        else:
            n_trials = 300 if model_name == "RidgeRegression" else 25
    if patience is None:
        if model_name == "XGBoost":
            patience = 5
        else:
            patience = 100 if model_name == "RidgeRegression" else 10

    if existing_result_index is not None:
        print(f"🔄 {model_name} on {dataset_name} has already been tuned. Retraining and updating results.")
    else:
        print(f"🚀 Starting {model_name} on {dataset_name} with {n_trials} trials")

    def objective(trial):
        if model_name == "RidgeRegression":
            return tune_ridge(trial, X, y)
        elif model_name == "RandomForest":
            return tune_rf(trial, X, y)
        elif model_name == "XGBoost":
            return tune_xgb(trial, X, y)

    early_stopping = EarlyStoppingCallback(patience=patience)

    # Progress bar wrapper
    with tqdm(total=n_trials, desc=f"{dataset_name} | {model_name}", leave=False) as pbar:
        def progress_bar_callback(study, trial):
            pbar.update(1)

        study = optuna.create_study(direction="minimize")
        study.optimize(objective, n_trials=n_trials,
                       callbacks=[early_stopping, progress_bar_callback], catch=
        (Exception,))

    best_params[(dataset_name, model_name)] = study.best_params

    # Retrain and evaluate
    if model_name == "RidgeRegression":
        model = Ridge(**study.best_params)
    elif model_name == "RandomForest":
        model = RandomForestRegressor(**study.best_params, random_state=42,
        n_jobs=-1)
    elif model_name == "XGBoost":
        model = XGBRegressor(**study.best_params, random_state=42, n_jobs=-1)

    y_pred = cross_val_predict(model, X, y, cv=kf, n_jobs=-1)

    y_true = np.expml(y) if dataset_name == "Log" else y
    y_pred = np.expml(y_pred) if dataset_name == "Log" else y_pred

    r2, rmse, mae, pmae = evaluate(y_true, y_pred)

    # Update or append the result based on whether it's already in results_tuned
    result = {
        "Dataset": dataset_name,
        "Model": model_name + " (Tuned)",
        "R²": round(r2, 4),
        "RMSE": round(rmse, 2),
        "MAE": round(mae, 2),
        "PMAE (%)": round(pmae, 2)
    }

    if existing_result_index is not None:
        # Update the existing result in results_tuned
        results_tuned[existing_result_index] = result
    else:
        # Append as a new result if it's not already present
        results_tuned.append(result)

    # Save the trained model
    os.makedirs("../models", exist_ok=True)
    model_path = f"../models/{dataset_name}_{model_name}.pkl"
    joblib.dump(model, model_path)
    print(f"💾 Model saved to {model_path}")

    print(f"✅ Finished {model_name} on {dataset_name}")

```

- ผลลัพธ์หลัง Train

```

import pandas as pd

# Convert the results list to a DataFrame
results_df = pd.DataFrame(results_tuned)

# Improved styled DataFrame for better header and width display
styled_results = results_df.style.format({
    "R²": "{:.4f}",
    "RMSE": "{:,.2f}",
    "MAE": "{:,.2f}",
    "PMAE (%)": "{:,.2f}"
}).background_gradient(subset=["R²"], cmap="Blues") \
.background_gradient(subset=["RMSE", "MAE", "PMAE (%)"], cmap="Reds_r") \
.set_table_styles([
    {"selector": "thead th", "props": [
        ("background-color", "#1976D2"),
        ("color", "white"),
        ("font-weight", "bold"),
        ("text-align", "center"),
        ("white-space", "nowrap"),
        ("padding", "10px")
    ]},
    {"selector": "tbody td", "props": [
        ("border", "1px solid #ddd"),
        ("text-align", "center"),
        ("padding", "8px"),
        ("white-space", "nowrap")
    ]},
    {"selector": "table", "props": [
        ("width", "100%"),
        ("table-layout", "fixed")
    ]}
]) \
.set_properties(**{"text-align": "center"})

styled_results

```

	Dataset	Model	R ²	RMSE	MAE	PMAE (%)
0	Drop_Log	RidgeRegression (Tuned)	0.6535	6,384.77	4,611.20	35.27
1	Drop_Log	RandomForest (Tuned)	0.8841	3,691.70	2,190.97	16.76
2	Drop_Log	XGBoost (Tuned)	0.9165	3,134.09	1,690.69	12.93
3	Drop_Iso	RidgeRegression (Tuned)	0.5603	6,691.75	4,817.97	37.80
4	Drop_Iso	RandomForest (Tuned)	0.8707	3,628.14	2,231.00	17.50
5	Drop_Iso	XGBoost (Tuned)	0.9068	3,080.38	1,820.88	14.29
6	Fill_Log	RidgeRegression (Tuned)	0.6900	7,444.66	5,264.39	28.39
7	Fill_Log	RandomForest (Tuned)	0.9091	4,030.28	2,233.45	12.04
8	Fill_Log	XGBoost (Tuned)	0.9394	3,291.68	1,591.68	8.58
9	Fill_Iso	RidgeRegression (Tuned)	0.6361	7,307.91	5,252.39	29.65
10	Fill_Iso	RandomForest (Tuned)	0.8600	4,532.30	2,572.74	14.52
11	Fill_Iso	XGBoost (Tuned)	0.9111	3,611.20	1,875.35	10.59

- heatmap ของค่า R^2 ของแต่ละ model

```

# Convert results_tuned to a DataFrame
results_tuned_df = pd.DataFrame(results_tuned)

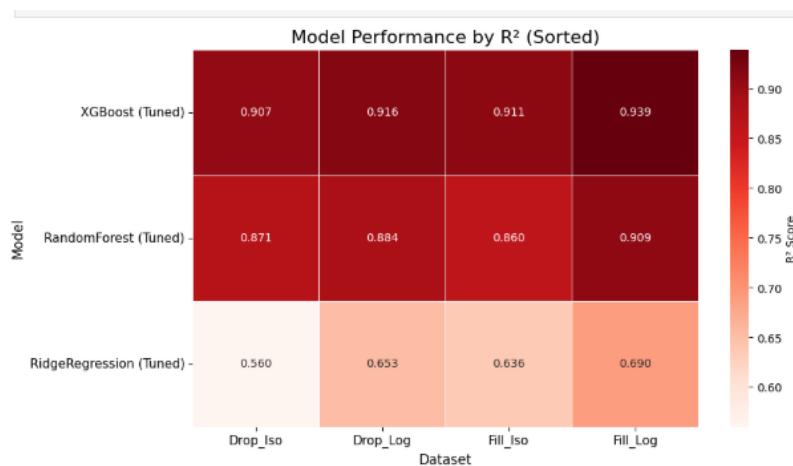
# Check if results_tuned_df is empty
if results_tuned_df.empty:
    print("No tuned results available to create a heatmap.")
else:
    # Create a pivot table
    heatmap_data = results_tuned_df.pivot(index="Model", columns="Dataset",
    values="R²")

    heatmap_data["Avg R²"] = heatmap_data.mean(axis=1)
    heatmap_data.sort_values("Avg R²", ascending=False, inplace=True)
    heatmap_data.drop(columns="Avg R²", inplace=True)

    # Red theme color map
    plt.figure(figsize=(10, 6))
    sns.heatmap(
        heatmap_data,
        annot=True,
        fmt=".3f",
        cmap="Reds",          # red theme
        linewidths=0.5,
        linecolor='white',
        cbar_kws={"label": "R² Score"}
    )

    plt.title("Model Performance by R² (Sorted)", fontsize=16)
    plt.xlabel("Dataset", fontsize=12)
    plt.ylabel("Model", fontsize=12)
    plt.xticks(fontsize=11)
    plt.yticks(fontsize=11)
    plt.show()

```



บทที่ 5

วิเคราะห์ผล

5.1 การเปรียบเทียบผลลัพธ์ระหว่างแบบจำลองที่ปรับและไม่ปรับค่าพารามิเตอร์

```

# Convert to DataFrame

df_no_tune = pd.DataFrame(results_no_hyper_tune)
df_tuned = pd.DataFrame(results_tuned)

df_no_tune.to_csv('../dataset/results_no_hyper_tune.csv', index=False)
df_tuned.to_csv('../dataset/result_tuned.csv', index=False)

# df_no_tune = pd.read_csv('../dataset/results_no_hyper_tune.csv') #use for called
# from saved result
# df_tuned = pd.read_csv('../dataset/result_tuned.csv')

# Clean up model/dataset names
df_tuned["Model"] = df_tuned["Model"].str.replace(r" \{Tuned\}", "", regex=True)
df_tuned["Dataset"] = df_tuned["Dataset"].str.replace(r" Selected", "", regex=True)

# Merge for comparison
comparison = pd.merge(
    df_no_tune,
    df_tuned,
    on=["Dataset", "Model"],
    suffixes=("_NoTune", "_Tuned")
)

# Compute differences
comparison["Δ R²"] = comparison["R²_Tuned"] - comparison["R²_NoTune"]
comparison["Δ RMSE"] = comparison["RMSE_NoTune"] - comparison["RMSE_Tuned"]
comparison["Δ MAE"] = comparison["MAE_NoTune"] - comparison["MAE_Tuned"]
comparison["Δ PHAE"] = comparison["PHAE (%_NoTune)"] - comparison["PHAE (%_Tuned)"]

# Round for display
comparison = comparison.round(3)

# Sort by R² improvement
comparison.sort_values(by="Δ R²", ascending=False, inplace=True)
comparison.reset_index(drop=True, inplace=True)

# Style the comparison DataFrame
styled_comparison = comparison.style.format({
    "R²_NoTune": "{:.4f}",
    "RMSE_NoTune": "{:,.2f}",
    "MAE_NoTune": "{:,.2f}",
    "PHAE (%_NoTune)": "{:,.2f}",
    "R²_Tuned": "{:.4f}",
    "RMSE_Tuned": "{:,.2f}",
    "MAE_Tuned": "{:,.2f}",
    "PHAE (%_Tuned)": "{:,.2f}",
    "Δ R²": "{:.4f}",
    "Δ RMSE": "{:,.2f}",
    "Δ MAE": "{:,.2f}",
    "Δ PHAE": "{:,.2f}"
}).background_gradient(subset=["R²_NoTune", "R²_Tuned", "Δ R²", cmap="Blues"] \
    .background_gradient(subset=["RMSE_NoTune", "MAE_NoTune", "PHAE (%_NoTune",
    "RMSE_Tuned", "MAE_Tuned", "PHAE (%_Tuned)", cmap="Reds_r"] \
    .background_gradient(subset=["Δ RMSE", "Δ MAE", "Δ PHAE"], cmap="Reds") \
    .set_table_styles([
        {"selector": "thead th", "props": [
            ("background-color", "#1976d2"),
            ("color", "white"),
            ("font-weight", "bold"),
            ("text-align", "center"),
            ("white-space", "nowrap"),
            ("padding", "10px")
        ]},
        {"selector": "tbody td", "props": [
            ("border", "1px solid #ddd"),
            ("text-align", "center"),
            ("padding", "8px"),
            ("white-space", "nowrap")
        ]},
        {"selector": "table", "props": [
            ("width", "100%"),
            ("table-layout", "fixed")
        ]}
    ]) \
    .set_properties(**{"text-align": "center"})

styled_comparison

```

	Dataset	Model	R ² _NoTune	RMSE_NoTune	MAE_NoTune	PMAE (%)_NoTune	R ² _Tuned	RMSE_Tuned	MAE_Tuned	PMAE (%)_Tuned
0	Fill_Iso	XGBoost	0.8310	4.984.69	3.024.49	17.07	0.9110	3,611.20	1,875.35	10.59
1	Fill_Log	XGBoost	0.8930	4.372.10	2.639.08	14.23	0.9390	3,291.68	1,591.68	8.58
2	Drop_Iso	XGBoost	0.8700	3,633.03	2,332.95	18.30	0.9070	3,080.38	1,820.88	14.29
3	Drop_Log	XGBoost	0.8920	3,564.99	2,216.08	16.95	0.9160	3,134.09	1,690.69	12.93
4	Fill_Iso	RidgeRegression	0.6360	7,307.91	5,252.39	29.65	0.6360	7,307.91	5,252.39	29.65
5	Drop_Iso	RidgeRegression	0.5600	6,691.75	4,818.04	37.80	0.5600	6,691.75	4,817.97	37.80
6	Drop_Log	RidgeRegression	0.6540	6,384.77	4,611.21	35.27	0.6540	6,384.77	4,611.20	35.27
7	Fill_Log	RidgeRegression	0.6900	7,444.66	5,264.40	28.39	0.6900	7,444.66	5,264.39	28.39
8	Drop_Log	RandomForest	0.8920	3,571.32	2,001.97	15.31	0.8840	3,691.70	2,190.97	16.76
9	Drop_Iso	RandomForest	0.8800	3,497.22	2,038.81	15.99	0.8710	3,628.14	2,231.00	17.50
10	Fill_Log	RandomForest	0.9250	3,666.07	1,735.70	9.36	0.9090	4,030.28	2,233.45	12.04
11	Fill_Iso	RandomForest	0.8900	4,011.87	1,999.36	11.29	0.8600	4,532.30	2,572.74	14.52

5.2 การเปรียบเทียบประสิทธิภาพของ model แต่ละประเภท

```

metrics = ["R2", "RMSE", "MAE", "PMAE (%)"]
melted = pd.melt(
    comparison,
    id_vars=["Dataset", "Model"],
    value_vars=[f"{m}_{sfx}" for m in metrics for sfx in ["NoTune", "Tuned"]],
    var_name="Metric_Version",
    value_name="Score"
)

# Split 'Metric_Version' into two columns: Metric and Version
melted[["Metric", "Version"]] = melted["Metric_Version"].str.extract(r"
(.*?)_(?:NoTune)|(?:Tuned)")
melted.drop(columns=["Metric_Version"], inplace=True)

# Plot
g = sns.catplot(
    data=melted,
    kind="bar",
    x="Score",
    y="Model",
    hue="Version",
    col="Metric",
    row="Dataset",
    palette="Set2",
    sharex=False,
    height=4,
    aspect=1.6
)

g.fig.subplots_adjust(top=0.9)
g.fig.suptitle("Model Performance Comparison: Tuned vs. NoTune")
plt.show()

```

Model Performance Comparison: Tuned vs. NoTune



บทที่ 6

สรุปผล

1. การเตรียมและการจัดการข้อมูล

การประยุกต์ใช้เทคนิค log transformation ควบคู่กับ IQR ส่งผลให้ประสิทธิภาพของ model ทุกประเภทพัฒนาขึ้นอย่างชัดเจน โดยเฉพาะอย่างยิ่งในกรณีของ XGBoost เนื่องจากชุดข้อมูลที่ใช้มีลักษณะการกระจายตัวแบบเบ้ การแปลงข้อมูลด้วยลอการิทึมจึงช่วยปรับการกระจายตัวของข้อมูลให้เข้าใกล้การแจกแจงแบบปกติมากยิ่งขึ้น

การทำ Feature engineering ด้วยเทคนิค Clustering ไม่ได้ส่งผลในการปรับปรุงประสิทธิภาพของชุดข้อมูลอย่างมีนัยสำคัญ เนื่องจากคุณลักษณะเหล่านี้จะถูก drop ออกในกระบวนการคัดเลือกคุณลักษณะในขั้นตอนถัดไปอยู่แล้ว ดังนั้นเราจึงสามารถข้ามกระบวนการ Clustering ได้โดยไม่ส่งผลกระทบต่อประสิทธิภาพโดยรวมของ model

2. การสร้างแบบจำลอง

จากชุดข้อมูลไม่มี linear relationships ที่ชัดเจน ส่งผลให้ linear-based models ที่ใช้มีประสิทธิภาพที่น้อยลง โดยในทางกลับกันแบบจำลอง Tree-based models มีความเหมาะสมมากกว่าเนื่องจากสามารถจัดการกับรูปแบบความสัมพันธ์ที่ไม่เป็นเส้นตรงได้อย่างมีประสิทธิภาพผ่านการแบ่งแบบ threshold-based decision

จากการ Train พบว่า XGBoost เป็นแบบจำลองที่ดีที่สุดหลังจากการปรับค่าพารามิเตอร์ โดยมีค่า R^2 อยู่ที่ 93.94% และมีค่า %MAE ที่ 8.58%

โดยสรุป หลังจากการจัดค่าผิดปกติแล้ว ชุดข้อมูลยังคงมีลักษณะเบ้ขวา ซึ่งทำให้ model มีความแม่นยำสูงในบริเวณที่ข้อมูลมีการกระจุกตัว และมีความแม่นยำน้อยลงในบริเวณที่ข้อมูลของการกระจายตัวน้อย